



Applied static analysis and specialization of cross-core syscalls for multi-core AUTOSAR OS

Gerion Entrup¹ · Andreas Kässens¹ · Björn Fiedler¹ · Daniel Lohmann¹

Accepted: 24 August 2024 / Published online: 3 October 2024
© The Author(s) 2024

Abstract

The development of static real-time control systems often follows a closed-world assumption, allowing extensive RTOS-aware whole-program optimization. For single-core systems, previous work could show the high potential of control-flow-aware static system-call tailoring. However, due to an exponential state explosion in the analysis phase, it cannot simply be extended to a multi-core setting, since the core's relative timing to each other is undetermined. In this work, we present MultiSSE, a multi-core capable and RTOS-aware static whole-system optimization. First, MultiSSE analyzes the system by determining the relative positions of multiple cores only when necessary. For that, it exploits structural control flow and optionally timing information to handle each core separately as much as possible. Based on the analysis result, a synthesis applies lock elision and system-call optimization to generate specialized multi-core real-time systems for AUTOSAR OS. To enable a static prediction of the run-time reduction, we additionally provide cost models for the optimized cross-core system calls and evaluate the approach with synthetic benchmarks and a real-world quadrotor application. MultiSSE was able to optimize or even completely elide costly cross-core system calls and system objects leading to a reduction of up to **14%** of a task's execution time. In this extended version of a conference publication (Entrup et al. 2023), we provide an advanced description, new cost models, and an end-to-end measurement by developing a synthesis complementing the analysis.

Keywords Real-Time Systems · Multi Core · Lock Elision · IPI avoidance · Syscall Cost Model

✉ Gerion Entrup
entrup@sra.uni-hannover.de

Andreas Kässens
kaessens@sra.uni-hannover.de

Björn Fiedler
fiedler@sra.uni-hannover.de

Daniel Lohmann
lohmann@sra.uni-hannover.de

¹ Systems Research and Architecture Group (SRA), Leibniz Universität Hannover, Appelstraße 4, 30167 Hannover, Germany

Abbreviations

ABB	Atomic basic block
BCET	Best-case execution time
BFS	Breadth-first search
CFG	Control-flow graph
cross syscall	Cross-core system call
IPI	Inter-processor interrupt
IR	Intermediate representation
ISR	Interrupt service routine
LABSS	Local-core abstract system state
MABSS	Multi-core abstract system state
MSTG	Multi-core state transition graph
[PMU	Performance Monitor Unit
RTCS	Real-time control system
RTOS	Real-time operating system
RTS	Real-time system
SP	Synchronization point
SSE	System-state enumeration
WCET	Worst-case execution time
WPO	Whole-program optimization

1 Introduction

RTCS are often embedded in a surrounding system to fulfill a special purpose (Marwedel 2006; Cooling 2003). As a consequence, all their code and hardware must be known before runtime, which enables comprehensive whole-program optimizations (WPOs) during compilation. This also applies to the RTOS. For example, as with OSEK/AUTOSAR-OS (OSEK/VDX Group 2005; AUTOSAR 2013), all possible events have to be mapped to threads, semaphores and interrupt service routines (ISRs) defined at compile time, allowing preallocation of memory and simpler management algorithms. But also the much newer Zephyr real-time operating system (RTOS) [6] offers the option of static initialization for the same purpose.

1.1 RTOS-Aware Static Analysis

Usually, the WPO optimizes the application without any knowledge of the semantics and thread model of the RTOS. Dietrich and colleagues, however, have shown (Dietrich et al. 2015, 2017) that there can be significant advantages in extending the analysis beyond the system call boundary and including the RTOS' semantics. If, for example, a corresponding optimization statically determines during the analysis of a system call `ActivateTask(B)` that `B` is set ready but never scheduled (because it has a lower priority than the task currently executing), it can specialize this system call in such a way that it never activates the expensive rescheduling code. Such system

call specialization can significantly reduce memory consumption, latency and the robustness of the optimized system (Dietrich et al. 2015, 2017).

The core of the specialization is an analysis, the system-state enumeration (SSE), which calculates all interactions between the application and RTOS by statically enumerating all RTOS states (state of threads, ISR, ...) that can exist during run-time. However, the SSE is designed exclusively for single-core systems. Despite the authors' claim that their approach also "would also work for multicore systems" (Dietrich et al. 2015), this statement is only true for the unrealistic assumption of a fully partitioned system, so that the code of each core never interacts with that of another.

1.2 About this Paper

In this paper, we present MultiSSE, an analysis that is able to analyze partitioned fixed-priority multi-core systems with cross-core interactions, such as described by AUTOSAR 4.0 (AUTOSAR 2013) (e.g., cross-core task activation or spinlocks). Additionally, we present a set of optimizations with belonging end-to-end measurements based on the analysis.

A complete enumeration of system states (as in the SSE) on multiple cores would result in calculating the cross product of single core states, leading to a state explosion (Rinard 2001; Miné 2015). Thus, the MultiSSE works by analyzing each core separately in the default case. *Only* when it reaches a cross-core interaction, it builds explicit synchronization points that manifest the relative execution positions of all involved cores.

Based on the analysis, we developed several optimizations that especially target multi-core systems: We present a method to prevent unnecessary inter-processor interrupts (IPIs) and avoid unnecessary locks. As part of that, we show fine-granular cost models that allow differentiation of the phases of multiple cross-core interactions to predict the expected run-time savings.

In particular, we claim the following contributions:

- We describe MultiSSE, an approach for the analysis and optimization of multi-core real-time systems at the application–RTOS interface.
- As part of MultiSSE, we provide a feasible algorithm for enumerating all possible RTOS states on a multi-core system together with an ordering of these states.
- To demonstrate the use of the algorithm, we propose possible system call specializations that have the potential to reduce jitter and priority inversion in event-triggered multi-core real-time systems.
- We implement a multi-core AUTOSAR-compliant system and automatically apply our proposed specializations to it during compilation.
- We model the cost of cross-core interactions in real-time systems and measure the optimization potential of our system specialization.

This paper is an extended version of our previous conference publication (Entrup et al. 2023), which already covered the first three contributions (the analytical part), but had to omit relevant details and explanations due to space constraints, so we could improve on the comprehensibility of this part (mostly in 2.2 and 3.3). The most significant extension, however, can be found in the latter two contributions, which

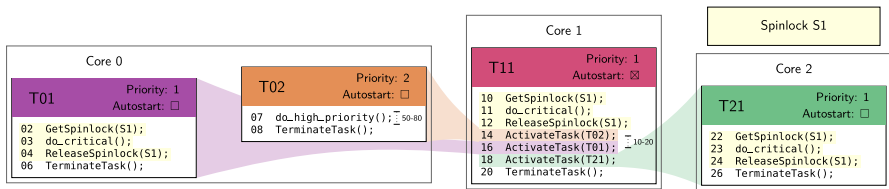


Fig. 1 Example application. Four tasks are distributed on three cores. Additionally, one spinlock exists. T11 starts automatically, the other tasks need an explicit activation which happens with the `ActivateTask` system call in T11. The yellow parts are critical sections they are guarded by protecting calls to the spinlock. The instruction numbers match the atomic basic blocks (ABBs) (Fig. 2, Sect. 3.1) (Color figure online)

describe the constructive part of the approach and the costs of cross-core interactions on real hardware (in detail 6.4, Sects. 7, 8.2, 8.3, 8.4 (partly), 8.5 (partly)). Readers who are familiar with the conference publication may want to continue directly in Sect. 6.

Since we make extensive use of acronyms across the paper, please find an overview at the end of the paper.

2 The MultiSSE Approach

The MultiSSE aims to provide detailed information about the application-RTOS interactions from its implementation (i.e., source code).

Its core assumption is that it is possible to statically detect all cross-core interactions at the system-call interface. For this, the underlying multi-core RTOS and the application code have to provide the following basic properties:

- (1) Deterministic scheduling policy, such as fixed-priority preemptive scheduling.
- (2) Partitioned scheduling, that is, each task (software- or hardware-triggered) is pinned to exactly one core and all potentially blocking/scheduling-relevant cross-core interaction takes place via system calls.
- (3) All system objects are known at compile-time.
- (4) All system calls are explicit, that is, their call instructions are detectable at compile-time including their actual parameter values. Indirect invocation by (nontrivial) function pointers is not allowed.
- (5) [Optional] For each system call and each computation block (code between two system calls), the worst-case and best-case execution times (WCET, BCET) are known.

In practice, requirements 1–4 are already fulfilled or easy to achieve for event-triggered hard real-time control systems as they are basically a technical consequence of predictability and, thus, already mandated by the dominant coding and RTOS standards of the domain: Examples include ARINC 653 (avionics), which prescribes global fixed-priority scheduling within its multi-core partitions (AEEC 2015) or multi-core AUTOSAR (automotive) (AUTOSAR 2013). Without loss of generality, we therefore exemplify our approach with the system model mandated by the AUTOSAR-OS standard.

2.1 A Brief Example

In Fig. 1, we exemplify some possible MultiSSE-based optimizations on an application using AUTOSAR OS. We will use this example (with four tasks partitioned over three cores) throughout the paper, hence briefly introduce it here: The application employs some cross-core system calls (cross syscalls) that may cause an action on another core: `GetSpinlock` and `ReleaseSpinlock` protect a critical region across multiple cores. `ActivateTask` sets a task ready, which might have been assigned to another core.

The structure of this application facilitates a number of system-call specializations, which all could be obtained by MultiSSE:

Initially, T11 on Core 1 is the only running task (autostart is set), Cores 0, 2 are idling. Hence, T11 does not need to obtain Spinlock S1 to get exclusive access to the critical section (lines 7–9), as it is guaranteed that neither T01 nor T21 (the possible competitors) are running at this stage. The respective system calls could be safely elided.

T11 continues by activating T02 and T01. Both are cross syscalls, which in the case of `ActivateTask` requires the calling core to issue an IPI to trigger a (potentially necessary) reschedule on the target core. In the case of T02 (line 10) the reschedule is necessary to wake up Core 0. In the case of T01 it depends: If T02 is still running, we could omit the reschedule (and, thus, the expensive IPI (Hammarlund et al. 2003)),¹ as T02 has priority over T01 and its `TerminateTask` in line 6 will issue a reschedule anyway. However, if T02 has already finished, Core 0 might again be idling and the IPI is required. By considering available best/worst-case execution time intervals, the MultiSSE can infer that the IPI can indeed be omitted. Additionally, in that case, the whole lock can be elided since all critical regions cannot interfere anymore.

So, in this example, the MultiSSE could detect three relatively expensive cross-core interactions (`Get/ReleaseSpinlock`, IPI in `ActivateTask`) that are infeasible by structure or timing.

2.2 MultiSSE: A Brief Overview

To provide detailed information about all application-RTOS interactions, the MultiSSE calculates the graph of all possible multi-core abstract system states (MABSSs). A MABSS represents the state of all CPU cores and RTOS objects at a specific point in time.

With existing techniques, it is possible to enumerate all core-local states (Dietrich et al. 2015). However, to use this in a multi-core setting, we need to respect the relative timing of the cores to each other. The naive approach to get all MABSSs is to enumerate all single-core states and then build the cross product between them. To avoid this combinatorial explosion, the MultiSSE calculates a reduced graph based on the fact that the cores do not interact arbitrarily with each other, but only at system-call sites that target other cores, which we call *cross syscalls*. Only at *these points*, the relative

¹ Additional to the cost tag of an IPI the (unnecessary) interruption on behalf of a lower-priority task is a case of rate-monotonic priority inversion (Leyva-del-Foyo et al. 2006; Hofer et al. 2009).

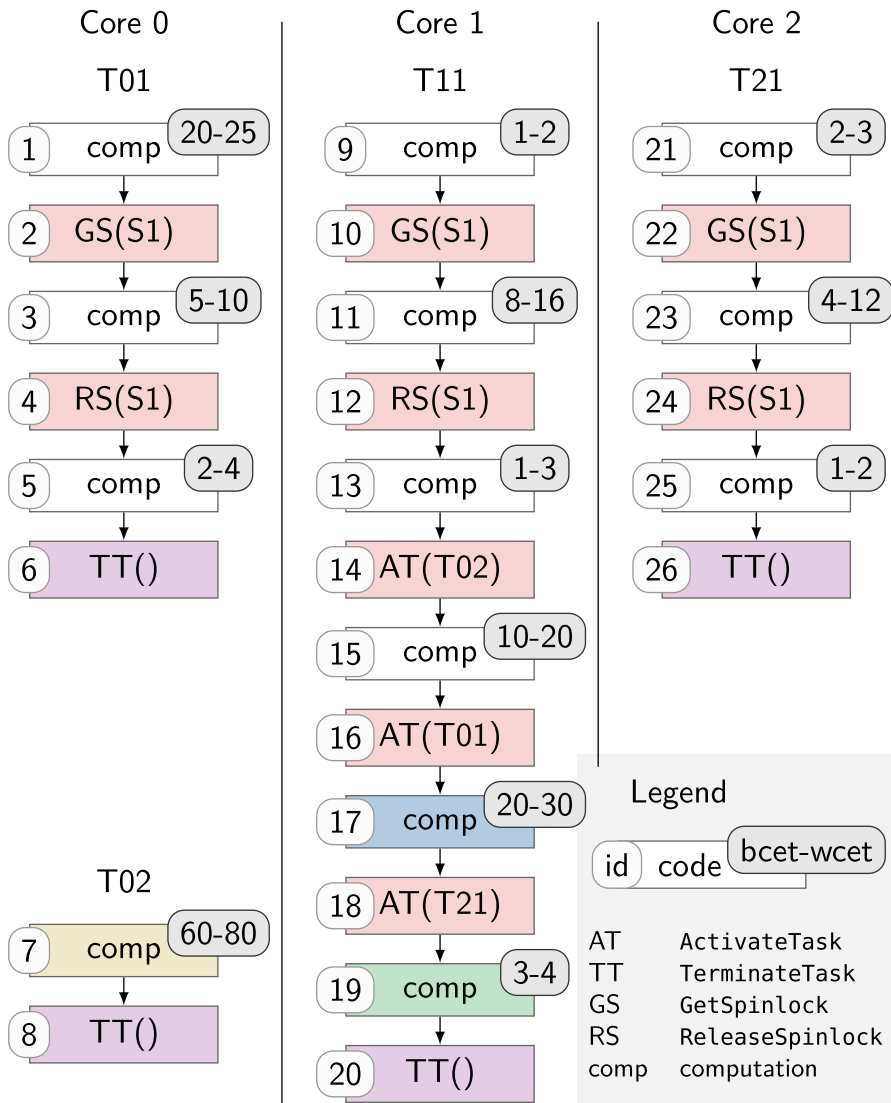


Fig. 2 ABBs of the example application. We can see a computation block (white) between each system call (in purple) or cross syscall (in red). The yellow, green, and blue blocks correspond to the equally colored blocks in Fig. 5 (Color figure online)

timings of only the *affected cores* are relevant. The MultiSSE analysis, therefore, constructs a smaller graph, the multi-core state transition graph (MSTG), that makes these interaction points explicit. It consists of two node types (both special forms of a MAbSS): Localcore abstract system states (LAbSSs) describe the current state of a single core, while SPs describe the current state of a set of cores together, which determines their relative position to each other. The main idea for the algorithm is to

```

1  def multisse():
2      # initialization
3      initial_sp = get_initial_sp()
4      queue = Queue(initial_sp)
5
6      while not queue.is_empty():
7          cur_sp = queue.pop()
8
9          # split_into_single_core internally creates "fork" edges
10         for labss in cur_sp.split_into_single_core():
11             # core-local analysis
12             cross_syscalls = single_core_analysis(labss)
13             for cross_syscall in cross_syscalls:
14                 # pairing-partner search
15                 pairing_partners, parent_sp, predecessor_sps = \
16                     pairing_partner_search(cross_syscall)
17                 # new entry SP construction
18                 new_entry_sp = create_sp(cross_syscall,
19                                         pairing_partners,
20                                         parent_sp,
21                                         predecessor_sps)
22                 # merge SPs and reevaluate if possible/necessary
23                 queue.push_reevaluations_if_necessary(new_entry_sp)
24                 if new_entry_sp.exists():
25                     continue
26
27                 # new exit SP construction
28                 for exit_sp in system_semantic(new_entry_sp,
29                                                 cross_syscall.cpu_id):
30                     connect(new_entry_sp, exit_sp, Type("global"))
31                 queue.push(exit_sp)

```

List. 1: High-Level algorithm of the MultiSSE (Pseudo-Code).

construct these two node types alternately. This happens by executing the following four steps, which Listing 1 also present as pseudocode:

- (1) **Initialization:** The system start represents the first SP (line 3). Since all cores start, the state of them relative to each other is known. (Example from Fig. 1: Evaluation starts with all cores in idle, except Core 1, which executes line 7 GetSpinlock.)
- (2) **Core-local analysis:** Follow the control flow on each core independently to calculate a set of LAbSS until a cross syscall occurs (line 12). (Example: Core 0 and 2 are idling, Core 1 is at the execution of GetSpinlock, a cross syscall.)
- (3) **Pairing-partner search:** For each cross syscall, calculate the list of all possible pairing partners, that is, all states in which all other affected cores may be at the same time (line 15-16). (Example: The lock is shared between all cores, so Core 1 needs to synchronize with Core 0 and 2. Therefore, the analysis has to find all states of Core 0 and 2 that may happen in parallel with Core 1. Normally, multiple pairing partners exist, in this case it is just the set of idle states of Core 0 and 2.)
- (4) **New SP construction:** These states form a new entry SP (line 18-21), which the analysis evaluates into one or more exit SPs by calculating the effects of the

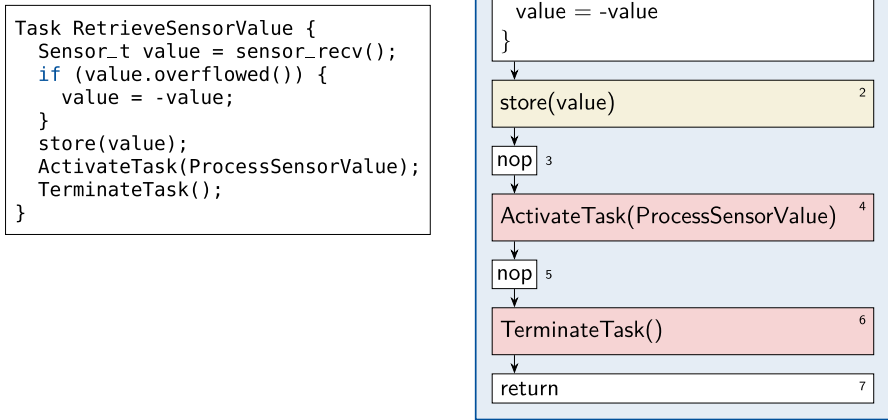


Fig. 3 Control-flow graph partitioned in ABBs. *computation* blocks are white, *call* blocks are yellow, *syscall* blocks are red. Note that `sensor_recv()` and `overflowed()` are not *call* blocks (unlike `store()`) since they neither call a system call directly nor a function that calls a system call (Color figure online)

cross syscall that leads to the SP (line 28-31). Starting at the introduced exit SPs, continue with step (2). (Example: The acquisition of the lock just goes through and does not change the idle status of the other cores, hence it results in one exit SP, which is the next start for core-local analysis.)

3 Detailed MSTG Construction

In the subsequent sections, we describe the MultiSSE construction in further detail.

3.1 Control-Flow Graph Preparation

To ease the actual analysis, we preprocess the control-flow graph by partitioning it into atomic basic blocks (ABBs) (Scheler and Schröder-Preikschat 2010). An ABB is a sequence of instructions that fulfills the following properties:

- (1) Every ABB has a single entry and a single exit instruction.
- (2) Every ABB is of the type *syscall*, *call*, or *computation*.
- (3) A *computation* ABB groups a sequence of instructions that are neither a system call nor a call to function that eventually issues a system call.
- (4) A *syscall* ABB contains a single system call instruction.
- (5) A *call* ABB contains a single call instruction.
- (6) Two sys/call ABBs do not follow each other, but always have a (potentially empty/NOP) computation ABB in between.

Hence, ABBs point out the system calls and summarize all other instructions as far as possible. For the analysis, each ABB represents a valid position in the code, that is the conceptual program counter (PC) of the abstract machine. For ABB construction, we use a dominator-based approach described in Dietrich et al. (2015); Dietrich (2019) (internally, each ABB is further grouped into basic blocks but this is not relevant for the MultiSSE). Figure 2 shows an example of the transformation of program code to ABBs.

The MultiSSE can optionally leverage timing information if the program's basic blocks come with a BCET and WCET. We map these times onto the ABBs. To further ease the analysis, sys/call ABBs are considered to happen atomically; we account for their times in the surrounding computation ABBs. Figure 3 displays the ABB-Graph of a sample task.

3.2 Data Structures and Terms

The core data structure of the MSTG is the *multi-core abstract system state* (MAbSS):

$$\begin{aligned} \text{MAbSS} &= (C, O) \quad C \subseteq \mathcal{C}, O \subseteq \mathcal{O}_g \\ \mathcal{C} &= \{c_0, \dots, c_{\max}\} \quad \text{set of all core contexts} \\ \mathcal{O}_g &= \{o_0, \dots, o_{\max}\} \quad \text{set of all global OS-object contexts} \end{aligned}$$

Fig. 4 visualizes one example. The context of an OS object is object-dependent (e.g., the spinning state of a spinlock). An OS object is global if it belongs to more than one core (i.e., involved cross syscalls). A core context is a six tuple:

$$\begin{aligned} c \in \mathcal{C} &= (\text{ABB}, \tau, \text{IRQ}, S, E, O) \\ \text{ABB} &= \text{the currently executed ABB (PC)} \\ \tau &= \text{the currently executed thread} \\ \text{IRQ} &= \text{interrupt enable bit (hardware state)} \\ S &= \text{call stack (path of called functions)} \\ E &\in \{\text{idle, waiting, normal}\} \quad (\text{execution state}) \\ O &= \text{set of core local OS-object contexts} \end{aligned}$$

So, a core context consists of its execution context (ABB, τ , and S), which defines its position in the control flow, its execution and IRQ state, which specifies the RTOS internal status of the core, and the set of core-local OS-object contexts, which describe the system objects bound to a specific core. The number of core contexts is variable: A MAbSS with exactly one core context forms a *local-core abstract system state* (LAbSS) that ignores the current state of all other cores:

$$\begin{aligned} \text{LAbSS} &= \text{MAbSS}_{|C|=1 \wedge O=\emptyset} \\ &= (C, O) \quad \text{with } |C| = 1 \text{ and } O = \emptyset \\ \cup_{\text{LAbSS}} &= \text{set of all LAbSSs} \end{aligned}$$

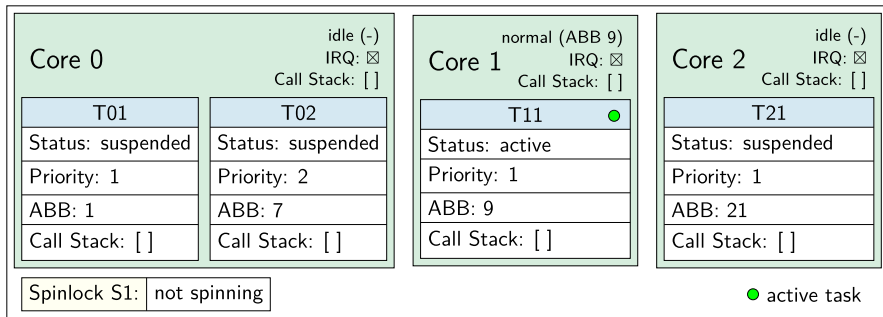


Fig. 4 The root SP of Fig. 1 as an example of a MAbSS. It contains several core contexts that describe their current execution state together with all RTOS-object contexts like the current task or Spinlock context. The root SP always contains all cores. A core context together with the assigned RTOS-object contexts for its own form a LAbSS (here each green part, respectively) (Color figure online)

A MAbSS with multiple core contexts forms a *synchronization point (SP)*, which determines the relative position of multiple cores to each other within the control flow:

$$SP = MAbSS_{C \neq \emptyset} = (C, O) \quad \text{with } |C| \geq 2$$

$$\cup_{SP} = \text{set of all SPs}$$

SPs are caused by a cross-core system call (cross syscall), a syscall ABB that interacts with another core. We call a LAbSS, which ABB is a cross syscall, a *cross-syscall LAbSS*. Since a cross syscall has a (potential) effect on each affected core, the originating LAbSS needs to be synchronized with LAbSSs on the affected cores, the *pairing partners*:

A *pairing partner* to a cross-syscall LAbSS is a LAbSS of an affected core that can possibly coexist in time with the originating LAbSS.

The analysis needs to determine the set of pairing partners for each affected core. The trivial upper bound for each of these sets is the set of *all* LAbSSs of the affected core. However, since the cross product of the pairing partner sets determines the number of inserted SPs, it is crucial to further constrain coexistence and, thereby, the number of pairing partners. We will discuss this later, but now define the MSTG:

$$\text{MSTG} = (V, E)$$

$$T_E = \{\text{local, global, fork, join}\}$$

edge type

$$V = \{\text{MAbSS}_0, \dots, \text{MAbSS}_n\}$$

$$E = \{(x, y, f) | (x, y) \in V^2, f = V \times V \rightarrow T_E\}$$

$$f(x, y) = \begin{cases} \text{local} & x \in \cup_{\text{LAbSS}}, y \in \cup_{\text{LAbSS}} \\ \text{global} & x \in \cup_{\text{SP}}, y \in \cup_{\text{SP}} \\ \text{fork} & x \in \cup_{\text{SP}}, y \in \cup_{\text{LAbSS}} \\ \text{join} & x \in \cup_{\text{LAbSS}}, y \in \cup_{\text{SP}} \end{cases}$$

The MSTG is a graph consisting of MAbSSs which are connected by edges. Each edge has a specific type depending on the connected node types. Local edges connect two LAbSSs and capture a core-local state transition. Global edges connect an entry SP with an exit SP and capture the transition caused by a cross syscall. Join edges connect a set of LAbSSs with the resulting entry SP, thus, capturing the pairing of a cross syscall state with its pairing partners. Fork edges connect an exit SP with the resulting set of LAbSSs when the analysis continues to calculate the effects of all cores individually.

For a better understanding, Fig. 5 illustrates an excerpt of the MSTG that results from the example (Fig. 1). The figure does not contain the initial state. It starts at SP a , which is based on the evaluation of `ReleaseSpinlock(S1)` (ABB 12). Core 0 and 2 are idling at this point. We will come back in detail to that example later.

3.3 The Algorithm

3.3.1 Initialization

The MultiSSE starts with the construction of the first SP, the *root SP*. It symbolizes the starting point of the scheduler on all cores and is therefore deterministic. In this state, all cores are in the idle state except the ones that should autostart a task according to the system specification. Additionally, the algorithm initializes all RTOS objects according to the system specification. Figure 4 shows the root SP of the example.

3.3.2 Single-Core Analysis

From the previous SP, the algorithm calculates the further processing of each core individually. It, therefore, splits the SP into LAbSS by extracting the specific local core context, adds fork edges from the SP to those LAbSSs, and omits all global object contexts.

The MultiSSE merges MAbSSs where possible. Therefore, if it has already evaluated the LAbSS in a previous iteration, it skips the state and its successors. Otherwise, it follows the ABB graph on each path by calculating a transition between two states

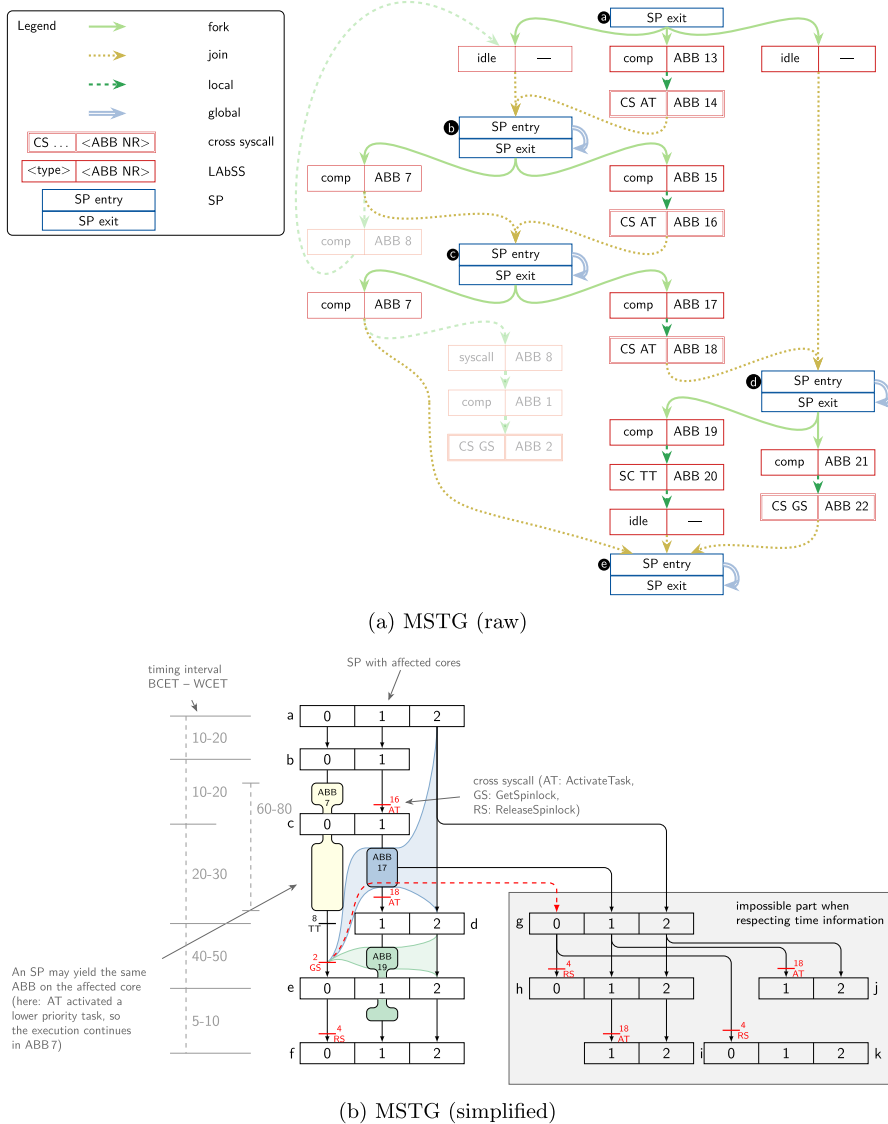


Fig. 5 An extract of the MSTG of the example (Fig. 1). In (a) the MSTG as described is illustrated, while (b) shows a simplified version. The simplified version summarizes the entry and exit SP and merges LABSSs, fork, local, and join edges. All red lines mark a state where a cross syscall happens. The yellow, blue, and green parts mark states where the core resides in the same ABB. Sometimes (for the yellow and green states), they are interrupted by an SP. The grey numbers describe the time interval between two SPs or the length of one ABB execution (Color figure online)

```

1  def single_core_analysis(labss):
2      queue = Queue(labss)
3      cross_syscalls = [] # return value
4      current_cpu = labss.get_cpu_id()
5
6      while not queue.is_empty():
7          cur_labss = queue.pop()
8          try:
9              next_labsss = system_semantic(cur_labss, current_cpu)
10             except IsCrossSyscall:
11                 cross_syscalls.append(cur_labss)
12                 continue
13             for next_labss in next_labsss:
14                 connect(cur_labss, next_labss, Type("local"))
15                 if mergable(next_labss):
16                     # the following states are evaluated
17                     merge_with_existent_labss(next_labss)
18                 else:
19                     queue.push(next_labss)
20
21     return cross_syscalls
22
23 def system_semantic(mabss, cpu_id):
24     active_abb = mabss.cpus[cpu_id].abb
25     if type(active_abb) == computation:
26         next_mabsss = follow_abb_chain(mabss)
27         return next_mabsss
28     if type(active_abb) == call:
29         return follow_call_chain(mabss)
30     if type(active_abb) == syscall:
31         if is_cross_syscall(active_abb) and not
32             mabss.contains_all_affected_cores(active_abb):
33             raise IsCrossSyscall
34     return interpret(active_abb) # syscall specific

```

List. 2: Pseudocode of the single-core analysis and the system_semantic function.

for every ABB (Listing 2, line 1). For that, it needs a transition function which it also uses for an SP-SP transition:

$$(\text{MAbSS}_{g+1,0}, \dots, \text{MAbSS}_{g+1,n}) = \text{system_semantic}(\text{MAbSS}_g, c)$$

g = generation
 c = the core to be evaluated

In the local-core case, c is always the only core context that is present in the LAbSS.

The `system_semantic` function (line 9) works dependent on the type of the ABB of the active core context c in the MAbSS: In the case of a computation ABB, it follows the control flow and constructs – for each follow-up ABB – a new LAbSS with the updated ABB for the active core (line 25–27). In the case of a call ABB, the analysis follows it by updating the core’s call stack and setting the new ABB of the called function (function return will pop the call stack again, line 28–29). For a syscall

```

1 def pairing_partner_search(cross_syscall_labss):
2     current_core = cross_syscall_labss.get_cpu_id()
3     affected_cores = get_affected_cores(cross_syscall_labss)
4     paths = get_parent_sps(cross_syscall_labss,
5                           union(affected_cores, current_core))
6     pairing_partners = list()
7     for path in paths:
8         parent_sp = path[0]
9         filtered_mstg = mstg.reachable_and_valid_for(path)
10        core_local_parent_sps = list()
11        for affected_core in affected_cores:
12            core_local_parent_sps.append(
13                get_core_local_parent(parent_sp, affected_core, current_core))
14        corridors = build_corridors(filtered_mstg, core_local_parent_sps)
15        for corridor in corridors:
16            # the corridor border is equivalent to its
17            # predecessor and successor SPs
18            for corridor_specific_pairing_partners in state_product(corridor):
19                pairing_partners.append(tuple(
20                    corridor_specific_pairing_partners,
21                    parent_sp,
22                    corridor.predecessor_sps))
23    return pairing_partners

```

List. 3: Implementation of the pairing-partner search MultiSSE (Pseudo-Code).

ABB, the analysis divides further: In the case of a cross syscall, the analysis stops the core-local analysis (line 31–33) and stores the LAbSS (line 10–12, here follows an SP). Otherwise, it interprets the system-call-specific effects on the state and emits one or multiple follow-up LAbSSs (line 34).

A single system call may produce multiple follow-up states. An example is `ReleaseSpinlock()`: When synchronizing 3 cores while 2 of them are spinning, the MultiSSE emits two follow-up MAbSSs, one for each of the two cores receiving the lock.

After executing `system_semantic`, the analysis merges the new LAbSSs with existing ones, if possible (line 15–17). For all remaining states, it iterates and applies the `system_semantic` function again (line 6,19). This step terminates if either all emitted new states are merged with previous ones (for example, the idle state has a transition to itself) or ends in a cross syscall. When terminated, the analysis continues with the next step.

3.3.3 Pairing-Partner Search

Once the MultiSSE detects a cross syscall, it searches for possible pairing partners. Listing 3 shows the process in pseudocode. First, it determines the affected cores, which are all cores the cross syscall synchronizes (line 3). Second, the analysis searches for LAbSSs on the affected cores that can exist at the same time. To not have to consider *all* LAbSSs as pairing partners, we take the last SPs into account. Every to-

be-synchronized core set has preceding SPs that synchronize at least the respective cores (that is, a superset).² We call an SP that precedes the cross-syscall LAbSS and synchronizes a superset of cores a *parent SP*:

A *parent SP* of an SP is an SP that synchronizes a superset of cores and for which exists at least one path in the MSTG between parent SP and SP that does not contain another parent SP.

Conceptually, the parent SP acts as a barrier in the control flow: The cross syscall in this context must happen after it, so the analysis can limit its search space to the graph part after a parent SP. Given the graph structure (caused by the state merging), multiple such parent SPs can exist that are reachable via disjoint paths (line 4). Because of that, the analysis also stores the chain of intermediate SPs leading to each parent and performs every following step for each path (line 7). For this search, the analysis uses an adapted breadth-first search (BFS).

For a better understanding, we want to use Fig. 5 as an example for such a search. In detail, we want to focus on the pairing process of `GetSpinlock(S1)` (ABB 2). This system call affects all three cores. Therefore, the analysis needs to find possible pairing partners for Core 1 and 2. The search for parent SPs leads to SP *a* which synchronizes Core 0, 1 and 2 with one path over SP *c* and SP *b*.

Since the MultiSSE merges MAbSS whenever possible, there may be loops to the parent SP or other paths that contradict the cross syscall (for example, an SP that synchronizes the same core). Therefore, the search first filters the graph (line 9): It removes all ingoing edges to the parent SP, thus, it eliminates circles. It also throws away every SP that either synchronizes the originating core and is not part of the path to the parent SP, or synchronizes only cores that are not affected by the cross syscall.

All SPs on the path to the parent SP form a relation in time between specific cores. Therefore, for each core, a later executed SP that synchronizes a subset of affected cores in the chain may mask the parent SP (it acts as a closer barrier for this reduced core set). It is enough to start the partner search from this point on. We name these SPs *core-local parent SPs* (line 12,13).

A *core-local parent SPs* of a pairing partner is the last SP on the path from the parent SP to the pairing partner that is also an element of the path from the parent SP to the cross-syscall LAbSS.

For example, SP *c* lies on the path from the parent SP *a* to ABB 2 and already synchronizes Core 0 with Core 1, so the search for pairing partners of Core 1 can start at SP *c*. For Core 2, the search must start directly at the parent SP, since all other SPs on the path do not synchronize Core 2.

When the analysis has calculated all core-local parent SPs, it continues to find pairing corridors (line 14): For every core, it iterates over all belonging states starting at the core-local parent SP. Each state can be restricted by following SPs, which may synchronize with another affected core and act as a barrier in its search area. As a consequence, the decision for a pairing area on one core can restrict the search space on the other cores, thus forming pairing corridors.

² Ultimately, the root SP synchronizes all cores.

```

1 def create_sp(cross_syscall, pairing_partners, parent_sp, predecessor_sps):
2     # construct SP
3     cpu_contexts = list()
4     for labss in chain(cross_syscall, pairing_partners):
5         cpu_contexts.append(labss.get_cpu_context())
6     global_context = parent_sp.get_global_context()
7     entry_sp = MAbSS(cpu_contexts, global_context)
8
9     # connect SP
10    connect(cross_syscall, entry_sp, Type("join"))
11    connect(pairing_partners, entry_sp, Type("join"))
12
13    return entry_sp

```

List. 4: Implementation of the SP construction MultiSSE (Pseudo-Code).

For example, by starting the search on Core 1, the algorithm finds the blue area first. All states in this area are possible pairing partners (here it is just ABB 17). The blue area is restricted by SP *d*, which synchronizes with Core 2, another core for which we want to find pairing partners, thus, restricting the search for Core 2 to the blue corridor. The SP *d* is a valid follow-up SP relative to SP *a* and SP *c*, so the green corridor also needs to be searched.

After determining a corridor, the analysis builds the product of each LAbSS in it (line 18 and following). For that, it only uses states that execute in a time interval, so computation or idling states.

The example contains just one state for each core on the corridor, so the cross syscall triggers two SPs: SP *e* (for the green corridor) and SP *g* (for the blue corridor).

3.3.4 SP Construction

With the cross syscall and the list of all pairing partners, the MultiSSE can create a new entry SP in the graph and connect it with all preceding LAbSSs (the cross syscall and its partners) via join edges. It therefore combines all core contexts of the preceding LAbSSs, which are disjunct by design, and takes the global contexts from the parent SP (Listing 4). Afterward, the analysis interprets the new entry SP with the `system_semantic` function, which results in a set of new exit SPs (Listing 1, line 28). For example, the interpretation of the `ActivateTask(T02)` (ABB 14) in SP *b* (entry) leads to SP *b* (exit), in which the state of T02 changes to "active" and the core executes ABB 7, the first ABB of this task.

The algorithm puts each exit SP in the work queue and starts the whole process again. Additionally, it determines the need for reevaluations, that is, existing SPs that need another evaluation (Listing 1, line 23):

- If the new SP has a predecessor in time that synchronizes another set or a superset of cores, the newly created state may add additional pairing partners for cross syscalls of this predecessor.

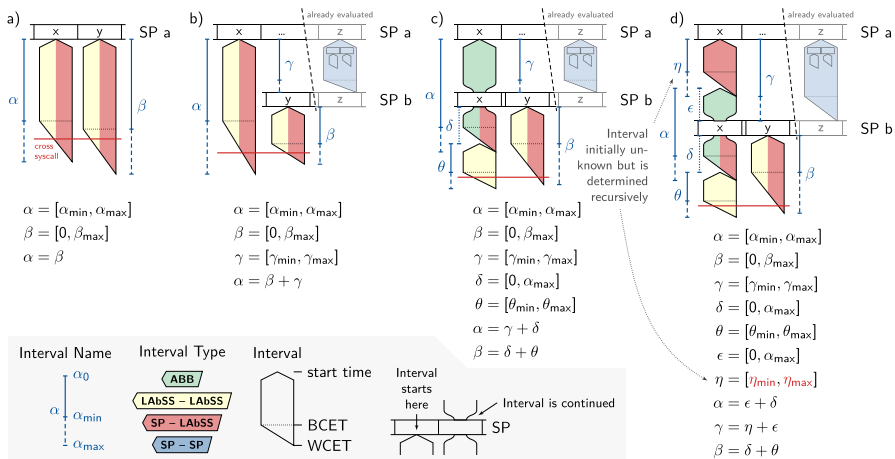


Fig. 6 All different building blocks for building the inequality system, sorted by their rising complexity. The to-be-compared intervals as well as the resulting inequality systems are shown. Some intervals have multiple types (indicated by their two colors). The requested interval is always that of the cross syscall to the last SP (the interval of β given all side constraints)

- If the to-be-created SP is already present and the algorithm just needs to add an edge between the old and the new SP, this adds a new system call order to the graph, so the SP needs to be reevaluated concerning this new edge.

4 Using Timing Information

The SPs in the MSTG describe an ordering of cross syscalls, since the graph connects them with edges that model an order in time. Each SP connects a cross syscall with a set of pairing partners, that is, other core states that can potentially run simultaneously. Naively, this would be all LABSS of the respective core, which we already have reduced by the described pairing partner search to those states that can actually coexist by structure (because they happen in the control flow after a previous SP).

In the following, we go further and also take execution times into account: At run time, many structurally possible combinations of pairing partners are actually impossible because of the times the cores spent in their different computation ABBs. To make use of this, the MultiSSE can leverage timing information already during construction: While searching for pairing partners, the algorithm immediately excludes combinations that are impossible because of timing constraints, that is, it neither creates the respective SPs nor evaluates them. This can drastically reduce the resulting number of SPs and, thus, analysis time.

In general, the algorithm works by comparing intervals on the different cores. For that, it assumes that each computation ABB has a BCET and WCET assigned. The idea of the MSTG is to synchronize a core only when needed (in an SP) and let it "run" independently until the next cross syscall. So, in an SP we know the exact point in time of different cores relative to each other. We can use that by just comparing the time

interval of the cross syscall relative to its preceding SP with the times of the potential pairing partners on the other cores (relative to the same SP). To compare the intervals, we build a linear inequality system, which is an equation system with additional interval constraints for each variable. Such a system forms a linear programming problem and can be solved efficiently with an LP-solver (Khachiyan 1979). For its functioning, the algorithm uses four types of time intervals:

ABB: The execution time interval of an ABB. This is a property of the ABB (Sect. 3.1).

L-L: A core-local execution time interval, given by a starting and ending LAbSS. The analysis calculates this interval type based on the underlying ABB intervals.

S-S: A time interval between two SPs. It describes all possible points in time in which the successor SP can exist relative to its predecessor.

S-L: A time interval starting at an SP and ending at a LAbSS that indicates an execution chain of one core relative to an SP. The pairing-partner search compares exactly these types of intervals, which the algorithm has to calculate.

For each interval, we will denote its starting time with t_0 , its minimum execution time with t_{min} and its maximum execution time with t_{max} .

4.1 The Inequality System

To build the inequality system, we can differentiate four different building blocks of rising complexity, which we illustrate in Fig. 6.

4.1.1 Simple intervals

Figure 6a shows the straightforward case. We see two different intervals on Core x and y, which are synchronized by SP a . Core x triggers a cross syscall, and we are interested in the information on whether the pairing partner on Core y can execute at the same time. We, therefore, want to compare the time interval α of SP a to the cross syscall with the time interval β of the LAbSS on core y starting at SP a (both of type **S-L**), which we need to calculate. In this case, Core x starts a LAbSS with a new ABB directly after SP a that eventually leads to the cross syscall (with possible LAbSSs in between). As a consequence, we need to calculate the interval α that starts and ends at a LAbSS (type **L-L** with inclusive start and exclusive end). Since all LAbSSs execute a single ABB which has a given BCET and WCET (type **ABB**), the problem reduces to a classical BCET or WCET analysis for which we chose a path-based approach (Holsti 2008; Ermedahl et al. 2011).

We need to compare α with the **S-L** interval β of the LAbSS of Core y that also starts at SP a . It denotes the time in which the LAbSS can take place. Therefore, β starts directly at SP a (time 0) and goes to the WCET of the LAbSS.

With the correct limits, we are now able to formulate the equation $\alpha = \beta$ and solve the inequality system. Thus, we obtain a new time interval that specifies the limits wherein the cross syscall with exactly this combination of pairing partners (and hence the resulting SP) can take place relative to SP a . To avoid later recalculations, we

store the resulting $\langle S-S \rangle$ interval as an attribute of a new edge between the SPs. If the inequality system has no solution, the SP is impossible.

4.1.2 Intervals with different SPs as a starting point

Figure 6b is mostly equivalent to Fig. 6a except that the pairing partner does not start at the same SP. So, to compare the intervals, we also need the $\langle S-S \rangle$ interval γ that describes the time between SP a and SP b . The algorithm has calculated this time interval already in a previous step: During the construction of SP b , it has built an inequality system whose solution specifies the interval γ . Therefore, it knows all needed interval limits and can compare them with $\alpha = \beta + \gamma$.

4.1.3 SPs that interrupt ABBs

In Fig. 6a and Fig. 6b all LAbSSs with their respective executing ABBs start directly at the SPs. However, often an SP interrupts the execution of an ABB and, thus, its $\langle ABB \rangle$ execution interval. For example, assume a cross syscall `GetSpinlock` that pairs with another core that does not hold the lock. In this case, the cross syscall does not change the LAbSS of the other core. Or assume an `ActivateTask` that activates a task on another core with a lower priority than the currently executing one. In that case, the LAbSS changes (the state context is different) but not the executing ABB. In these cases, some time of the $\langle ABB \rangle$ interval may have passed before the SP which the algorithm has to respect.

Figure 6c models exactly that. The $\langle ABB \rangle$ interval α is the time interval of an ABB whose execution is interrupted by SP b . We are interested in the $\langle S-L \rangle$ interval between SP b and the cross syscall which is a combination of the $\langle L-L \rangle$ interval θ and δ , the remaining part of α . In this case, the ABB execution starts directly at SP a , so we can calculate δ as $\alpha - \gamma$ which is the $\langle S-S \rangle$ interval between SP a and SP b and already calculated.

4.1.4 Interrupted ABBs that do not start at an SP

Figure 6d illustrate the most complex case. Here, the execution of the interrupted ABB does not start directly at an SP but is preceded by other LAbSSs. So, to be able to calculate δ , we need to know the length of the first part of α (ϵ) which, in turn, is the result of $\gamma - \eta$. γ is already calculated η is unknown. However, this maps to the same problem again but with another (prior) SP set and can be calculated recursively. The algorithm stops eventually (when it reaches the root SP). But, we can optionally terminate earlier by falling back to the previous execution time of an interrupted ABB as zero or its WCET (the ABB was executed in complete or not at all).

4.2 Challenges

For the ABB timings, we assume atomic system calls by accounting for the actual execution time in the surrounding computation ABBs. However, this assumption does

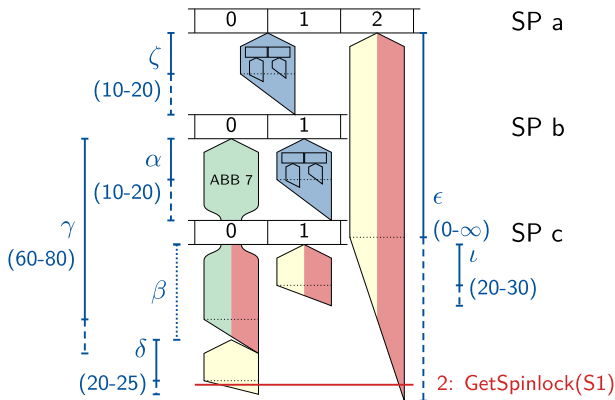


Fig. 7 Relevant parts of Fig. 5 for interval calculation of the blue corridor (Color figure online)

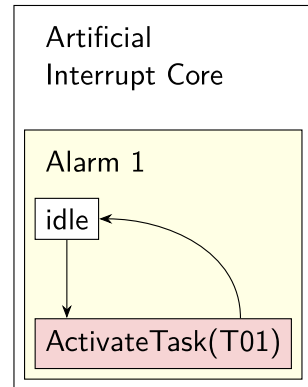
not hold for ABBs of the affected cores. Since the cross syscall triggers an IPI, the affected core is interrupted in an unknown computation ABB, which the SP makes explicit. However, the IPI handling time on the affected core needs to be accounted for as well. For this, our current implementation simply uses an additional constant term (i.e., the WCET of all IPI handlers) in the inequality system for every affected core. In future work, we want to support syscall-specific timing overheads.

Another challenge is handling loops. Given a cross syscall that executes within a loop, the affected cores could be interrupted multiple times, maybe even in the same ABB. This forms a loop in the MSTG, which the pairing partner search has to respect. When using the simple paring-partner search (without timing information), we can just ignore the loop: We are only interested in the fact, *whether* a synchronization is possible, not *when*. But for the paring-partner search with timing information, we need to determine the number of traversals. To remain sound, we therefore disable timing calculations for this particular search space whenever we detect an unbounded loop, that is, we set the time frame to $[0, \infty]$. However, commonly, loops are bounded in hard real-time systems. A bounded loop can be virtually unrolled, thus, omitting the problem altogether.

4.3 A Concrete Example

To illustrate the technique, we want to build a full inequality system of one part of the example (Fig. 7). We are using `GetSpinlock(S1)` (ABB 2) with the pairing partner of the blue corridor (ABB 17). The MultiSSE first calculates the time interval to the last SPs. This results in the interval $\delta = [20, 25]$ for the time between the end of the state that executes ABB 7 and the state that executes ABB 2, the interval $\iota = [0, 30]$ for the execution of the state that executes ABB 17 relative to SP c, and the interval $\epsilon = [0, \infty]$ for the time of the idling state of Core 2 starting from SP a. The time intervals between SP a, SP b, and SP c are already evaluated in previous iterations (the times are displayed in grey in Fig. 5). We further need the remaining time of the state that executes ABB 7 relative to SP c (case c). ABB 7 (interval γ)

Fig. 8 Artificial control flow for a simple periodic interrupt. Here, the BCET and WCET of the idle block represent the interrupt arrival time



starts its execution directly after SP b , so we can build the equation by comparing it with the interval between SP b and SP c (α). This results in the following inequality system:

$$\begin{array}{ll}
 \alpha + \beta = \gamma & \\
 \beta + \delta = \theta & \text{(time between SP } b \text{ and ABB 2)} \\
 \theta = \iota & \text{(compare Core 0 with Core 1)} \\
 \theta + \zeta + \alpha = \epsilon & \text{(compare Core 0 with Core 2)} \\
 \alpha = [10, 20] & \text{(time between SP } b \text{ and SP } c) \\
 \beta = [0, 80] & \text{(remaining execution interval)} \\
 \gamma = [60, 80] & \text{(BCET and WCET of ABB 7)} \\
 \delta = [20, 25] & \text{(ABB 7 to ABB 2)} \\
 \iota = [0, 30] & \text{(execution of ABB 17)} \\
 \epsilon = [0, \infty] & \text{(idling state on Core 2)} \\
 \zeta = [10, 20] & \text{(interval from SP } a \text{ to SP } b)
 \end{array}$$

This equation system has no solution: β evaluates to $[40, 70]$, leading to $[60, 95]$ for θ , which renders $\theta = \iota$ impossible. As a consequence, it eliminates the whole right part of the MSTG (the marked grey area), which leads to all cases where the lock spins which – as a result – is useless and can be elided. For the whole MSTG resulting from the presented example, the analysis generates 45 SPs without considering timing information and 12 SPs including timing information.

5 External Interrupts

Even though not mentioned explicitly, the algorithm is already prepared for dealing with external interrupt sources: From the perspective of the affected core, a cross syscall is an external interrupt (technically received as an IPI).

This allows us to model other external interrupt sources in exactly the same way: Each external interrupt source gets a virtual core assigned that runs a virtual interrupt program, a set of virtual ABBs that trigger the interrupt in the form of a cross syscall according to its specification. If the interrupt has inter-arrival times specified (e.g., as in AUTOSAR AUTOSAR 2013), we map this to an ABB with respective BCET and WCET interleaving the interrupt cross syscalls. The virtual interrupt cores start their execution at the system start or after an activating system call (e.g., for alarms). Figure 8 displays a virtual interrupt core for a simple periodic interrupt. The actual interrupt handling routine is done as part of the `system_semantic` function, which schedules the core context to the correct entry point. Periodic interrupts lead to a loop in the virtual interrupt program, which is unbounded without further information, resulting in a similar problem when respecting timing information as with other loops (discussed above). However, by giving the interrupt's inter-arrival times, we can constrain the maximum number of triggers within a hyperperiod, thus bounding the loop again.

Interrupts are a significant SP source, since, in theory, every interrupt could arrive at "any time". Modeling them all can result in a state explosion. In practice, however, plenty of them are likely to be impossible because of logical or timing constraints. The complexity of our algorithm scales with the number of SPs, so omitting impossible ones is desirable. We therefore optionally support specifying a task set belonging to an interrupt and omitting the interrupt while any of such tasks are executed to support the interrupt's "logic of action": An interrupt should not interrupt its own handling routine. Even though not yet completely supported in our implementation, our approach (to model interrupt sources as normal programs and interrupt triggering as cross syscalls) makes it possible to integrate additional (arbitrary complex) trigger conditions into the analysis to further reduce the number of SPs: For instance, it would be relatively easy to specify with such an artificial interrupt control flow that the send-buffer-empty interrupt of a communication device may only occur 10-20 msec after a `SendBuffer` invocation or that a button-released IRQ may not happen before a button-pressed. The same holds for platform-specific hardware behavior: If the interrupt controller supports advanced features, such as IRQ multicasts or round-robin or priority-based routing of IRQs, this could be incorporated into the implementation of the virtual cross syscall and its set of affected cores.

6 Applications

The MSTG represents all possible system states, which provides the knowledge to perform partial system-call specialization, also in multi-core settings. Such specialization consists of two further steps:

- (1) Extract the correct knowledge from the MSTG.
- (2) Actually perform and apply the optimization for the subsequent code generation.

We want to discuss the first step here on a variety of optimizations. First, the MultiSSE is capable of giving knowledge for single-core optimizations: Since the algorithm builds upon the single-core SSE, we designed the MSTG to be compatible, so such optimizations remain applicable. Second, we developed two additional multi-core

specific optimizations, which we also implemented and evaluated: Avoid unnecessary IPIs and locks. Here, we want to describe these optimizations in more detail and show how to extract the knowledge from the MSTG.

6.1 Single-Core Systems and Single-Core Phases

One core principle of our analysis is to perform single-core analyses for each core and let them interact only when necessary. This results in an MSTG that matches the single-core specific graph of the SSE enabling all its optimizations: In particular, these are pre-calculating core local scheduler decisions up to a static lookup table (Dietrich and Lohmann 2017), applying fault detection mechanisms to prevent transient hardware faults and laying the base for more advanced techniques like tightening the worst-case energy consumption or improving the memory footprint through efficient stack sharing (Dietrich and Lohmann 2018; Wägemann et al. 2018). Pre-calculating core local scheduler decisions and efficient stack sharing is not meaningfully adaptable to work across cores but remains functional in a multi-core environment. Applying additional information for fault detection and tightening the worst-case energy consumption should be extendable to work across cores and may give additional benefits. Since related work already synthesizes these optimizations, we concentrate on the multi-core specific ones.

6.2 Unnecessary IPIs

If an `ActivateTask` activates a task on another core, the standard RTOS implementation will set the target task state to "ready" and trigger an IPI to cause a reschedule on the affected core. However, the costly IPI is not necessary if we know for sure that a task with a higher priority runs on the affected core. A possible strategy to take this into account is to fall back to a global scheduler with a global kernel lock, so that the issuing core can safely determine which task is in execution on the remote core (Béchenec et al. 2006). This, however, comes at the price of no kernel-level parallelism and, thus, high pessimism regarding the kernel's WCET. The MSTG gives us the knowledge to do this less costly: If no SP originating from an `ActivateTask` leads to the execution of this task in the resulting MAbSS, we can safely omit the IPI.

For example, the `ActivateTask(S01)` (ABB 15) builds a single SP. This SP only interrupts the execution of ABB 7, thus rendering the IPI here useless.

The usage of `SetEvent` may also trigger an unnecessary IPI: If a task waits for an event that has a higher priority than the currently running one and that event is set from another core, an IPI is necessary to trigger a reschedule. In all other cases, the event just sets a flag in the task context. In the MSTG we are able to detect these cases as well: If an SP that originates from a `SetEvent` constructs a MAbSS with a waiting state in a task and this specific task is running and not waiting in the following MAbSSs anymore, we cannot avoid the IPI.

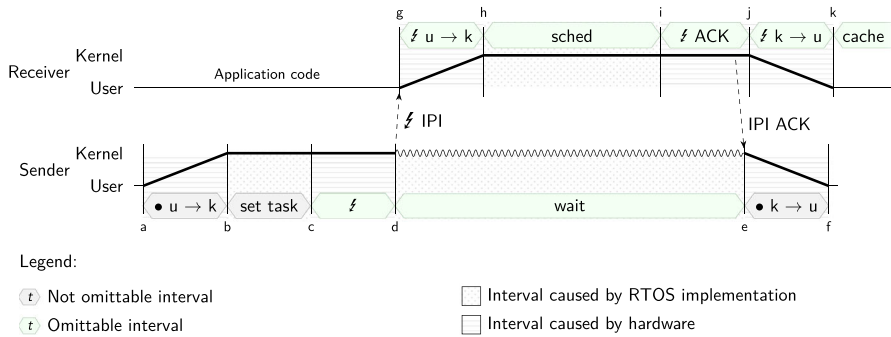


Fig. 10 Cost model of an avoidable inter-processor interrupt: A sender executes a system call `ActivateTask` or `SetEvent` that causes the RTOS kernel to trigger an IPI that a receiver core has to handle. The figure displays all necessary phases of the system call processing on both cores: Go into kernel mode, modify the scheduler state, issue an IPI, get interrupted on the other core, reschedule, acknowledge the IPI, and go back to user mode on both cores. All green phases are avoidable if we can prove the IPI unnecessary. On the receiver core may be additional costs due to cache misses *after* the interruption when continuing the normal application control flow (Color figure online)

6.4 Cross-Core–Interaction Cost Models

The MultiSSE gives us the knowledge which IPI and lock we can omit, but so far we do not know the resulting cost savings. They depend on two highly variable factors: First, the amount of IPIs and locks is entirely application-specific, so an overall benefit is impossible to measure in a generic case. Second, the saving per IPI and per lock depends on the underlying hardware architecture and RTOS implementation. Therefore, we developed models of the different execution phases for an (unnecessary) IPI and a lock, so we can predict the overall cost savings given the actual timings of the phases.

For that, we split each related system call into logically connected execution phases and show which exact phases we can omit with the optimization. Our assumption in the process is that the application comes together with deadlines, each specified with a BCET and WCET. Therein, only the WCET contains system-call execution times. We assume every system call to have a run time of zero up to its specific WCET (with the WCET assuming a system without cache). Our optimizations improve system-call execution times and, thus, the WCET, so they reduce pessimism. As part of the cost model, we developed different formulas to calculate that reduction. In our evaluation, we provide concrete numbers for each phase (also assuming a cold cache). These number gives an insight into how beneficial an optimization would be without actually performing and measuring it. Since all predictions assume a cold cache, an end-to-end measurement on a system with a warm cache will lead to smaller savings per omitted system call (due to caching), but, also to an overall reduced run time than estimated by the cold cache assuming WCET.

6.4.1 Cost Model for Avoidable IPIs

Figure 10 displays our IPI-model. It shows the end-to-end latency of an IPI that is an `ActivateTask` or `SetEvent` system call causes for the two involved cores differentiated by the necessary actions that the two cores need to perform.

First, the application on the sender core triggers the system call and causes a switch from user mode to kernel mode in $\langle \bullet u \rightarrow k \rangle$. After that, the sender sets the scheduler state on the receiver core in $\langle \text{set task} \rangle$ (set the task ready for `ActivateTask` or set the event flag for `SetEvent`). The next step is to trigger the actual IPI in $\langle \ell \rangle$. Since the AUTOSAR standard mandates the respective system calls to be synchronized, the sender needs to wait for the receiver core to acknowledge the IPI in $\langle \text{wait} \rangle$. Finally, it switches back to the user mode in $\langle \bullet k \rightarrow u \rangle$.

On the receiver core, the IPI causes a (hardware-induced) switch from user to kernel mode in $\langle \ell u \rightarrow k \rangle$. After that, the interrupt handling starts in $\langle \text{sched} \rangle$ (for the respective system calls this triggers a rescheduling). The receiver then acknowledges the IPI to the sender core in $\langle \ell \text{ACK} \rangle$ and switches back to user mode (including the dispatch to the new task, if any) in $\langle \ell k \rightarrow u \rangle$. Omitting the IPI means eliminating phase $\langle \ell \rangle$ and $\langle \text{wait} \rangle$ on the sender core. More critically, phases $\langle \ell u \rightarrow k \rangle - \langle \ell k \rightarrow u \rangle$ cause a priority inversion on the receiver core (Lakshmanan et al. 2010, 2011), which we can avoid entirely when omitting the IPI. The duration of most of the phases depends (mostly) on the hardware (phases $\langle \bullet u \rightarrow k \rangle$, $\langle \ell \rangle$, $\langle \bullet k \rightarrow u \rangle$, $\langle \ell u \rightarrow k \rangle$, $\langle \ell \text{ACK} \rangle$, $\langle \ell k \rightarrow u \rangle$), leaving no optimization potential for the RTOS developer. Only phases $\langle \text{set task} \rangle$ and $\langle \text{sched} \rangle$ depend on the scheduler and, thus, on the actual RTOS implementation.

During the IPI handling, cache misses can occur, specifically in phase $\langle \text{set task} \rangle$ and after $\langle \bullet k \rightarrow u \rangle$ on the sender core (not optimizable), but also in phase $\langle \text{sched} \rangle$ and in $\langle \text{cache} \rangle$ on the receiver core (optimizable by avoiding the IPI).

For the actual formula, we distinguish three cases:

- (1) **per IPI:** The time reduction for a single avoidable IPI.
- (2) **per task:** The time reduction for all avoidable IPIs issued during a single task. That means, the reduction in pessimism that results from the difference between the amount of IPIs that can potentially occur during a single (high priority) task execution (and thus are present in the WCET) in contrast to the amount of IPI that can actually occur as proven by our analysis.
- (3) **per core:** The time reduction for all avoidable IPIs of the task set that is executed on the specific core.

We additionally distinguish between sender and receiver core:

$$\text{Sender: } \Delta t_{\text{opt}} = \sum_i^{\# \text{tasks}} n_{\ell, i} \cdot \overbrace{\left(\overbrace{\langle \ell \rangle + \langle \text{wait} \rangle}^{\text{per IPI}} \right)}^{\text{per task}} \quad (1)$$

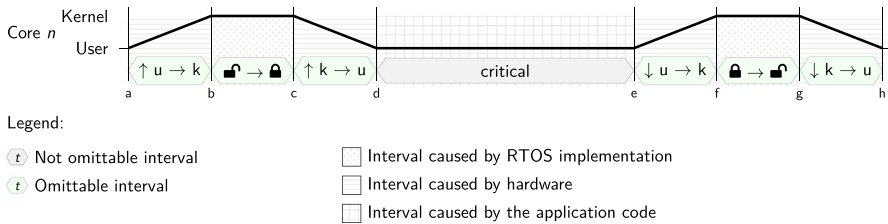


Fig. 11 Cost model of a critical region protected by a lock. In this case, the application issues `GetSpinlock`, gets the spinlock immediately, executes the critical region and calls `ReleaseSpinlock`. Here, all phases are shown separately (go into kernel mode, perform the (un)locking operation, and go back into user mode). Green phases are avoidable if we can prove that the lock never spins for this system-call site

$$\text{Receiver: } \Delta t_{\text{opt}} = \sum_i^{\# \text{tasks}} n_{f,i} \cdot \underbrace{\underbrace{\langle \uparrow u \rightarrow k \rangle + \langle \text{sched} \rangle + \langle \uparrow \text{ACK} \rangle + \langle \uparrow k \rightarrow u \rangle}_{\text{per IPI}}}_{\text{per task}} \quad (2)$$

per core

$\# \text{tasks}$: Number of tasks for this core. $n_{f,i}$: Amount of avoidable IPIs for task i .

6.4.2 Cost Model for Avoidable Locking Operations

Figure 11 shows our model for locking operations. It displays a critical region protected by a lock on a single core. The application first calls `GetSpinlock` which causes a switch to kernel mode in $\langle \uparrow u \rightarrow k \rangle$. After that, it acquires the lock in $\langle \uparrow k \rightarrow \text{lock} \rangle$ and switches back to user mode in $\langle \uparrow k \rightarrow u \rangle$ to execute a critical region $\langle \text{critical} \rangle$. When the critical region ends, the application issues `ReleaseSpinlock` which again causes a switch to kernel mode in $\langle \downarrow u \rightarrow k \rangle$, the lock releasing in $\langle \text{lock} \rightarrow \uparrow k \rangle$ and a switch back to user mode in $\langle \downarrow k \rightarrow u \rangle$.

For locks, we can perform two optimizations: Omit a single system call (phases $\langle \uparrow u \rightarrow k \rangle - \langle \uparrow k \rightarrow u \rangle$) or omit the system calls *and* the entire lock data structure. In this model, phases $\langle \uparrow k \rightarrow \text{lock} \rangle$ and $\langle \text{lock} \rightarrow \uparrow k \rangle$ depend on the actual lock implementation and, thus, the RTOS implementation, while phases $\langle \uparrow u \rightarrow k \rangle$, $\langle \uparrow k \rightarrow u \rangle$, $\langle \downarrow u \rightarrow k \rangle$, and $\langle \downarrow k \rightarrow u \rangle$ are hardware dependent.

Since all locking operations have to occur pairwise, we can build a single formula for each avoidable `GetSpinlock` system call together with the belonging `ReleaseSpinlock` system call. We use the same system as for the IPI cost calculation:

$$\Delta t_{\text{opt}} = \sum_{i=0}^{\# \text{tasks}} n_{\uparrow \downarrow, i} \cdot \underbrace{\left(\underbrace{\left(\underbrace{\uparrow u \rightarrow k + \downarrow u \rightarrow k}_{\text{per lock}} + \underbrace{\uparrow k \rightarrow u + \downarrow k \rightarrow u}_{\text{per lock}} \right)}_{\text{per task}} \right)}_{\text{per core}} \quad (3)$$

$\# \text{tasks}$: Number of tasks for this core.

$n_{\uparrow \downarrow, i}$: Amount of avoidable locking operations for task i .

7 Synthesis

To demonstrate the applicability of the optimizations enabled by the MultiSSE, we have realized them as part of a complete AUTOSAR system synthesis based on the *dOSEK* (Hoffmann et al. 2015) system generator. Similar to *dOSEK*, our system generation consists of three parts:

- (1) The application-independent AUTOSAR system libraries need to be implemented for the target architecture. Because *dOSEK* only supports single-core applications, we have extended it to a multi-core AUTOSAR library for two different architectures. Overcoming the proof of concept implementation for POSIX, we support the ARMv7 architecture by targeting a Raspberry Pi 4B with four Cortex-A72 cores. As recommended by the AUTOSAR specification (AUTOSAR 2013), we use the ARM privilege modes to protect the system and implemented hardware-specific components like the interrupt controller for IPIs, MMU configuration, caching, core-local timers and alarms according to the AUTOSAR specification requirements. Agnostic of the architecture, the scheduler is instantiated per core, using atomic *ready* and *event* flags for each task. With this design, we do not need global locking on the scheduler like Trampoline OS (Béchenne et al. 2006), but we are still able to atomically update the scheduler on cross-core interactions. As described in Fig. 10, we can therefore activate a task or set an event in two independent steps: setting the task's *ready* or *event* flag from any core, and triggering a reschedule on the core where the task is bound to.
- (2) Optimizing individual system calls requires specialization of their call sites. To achieve this, these sites in the application code are replaced during compilation with specific variants. We specialize the system calls on LLVM intermediate representation (IR) code, after the ABB partitioning (see Sect. 3.1). This way, each system call can be specialized to an implementation per ABB that may omit unnecessary IPIs or even may be completely removed due to the lock elision.
- (3) We generate application-specific code by parsing the system description and iterating over the system calls in the application. All defined OS objects like tasks, events, resources and spinlocks are instantiated statically, and the specialized system calls are generated for the application.

These three components are linked together to form the optimized executable.

8 Evaluation

To evaluate the findings of our approach and the resulting freedom for system-call specialization, we implemented the MultiSSE in ARA,³ a whole system optimizer with a focus on RTOS specialization based on LLVM (Fiedler et al. 2021; Lattner and Adve 2004). We implemented the algorithm in more than 5000 LoC of Python/C++ (including 326 for the SSE core, 1,822 specific for the MultiSSE, and 1,523 for the AUTOSAR model). The AUTOSAR libraries for ARM needed another 3,132 LoC in C++ (compared to 2,812 in *dOSEK*) and 878 in Python for the code generator.

For the evaluation, we apply the MultiSSE onto several systems in various depths:

- (1) Analysis tests: We apply the analysis on synthetic benchmarks and a set of AUTOSAR conformance tests (taken from Trampoline OS Béchenne et al. 2006) to prove the operability of the analysis. Thereby, we analyze the reduction of possible spinning states for the spinlocks, avoidable IPIs and surely set events. We compare the outcome of the MultiSSE with and without timing information against the plain information from the system specification. These tests are without the synthesis part and show the analysis' working.
- (2) Microbenchmarks: We measure the savings per phase in the IPI model and the lock model to get real-world numbers for the prediction formulas to be able to use them in the following synthesis tests.
- (3) Synthesis tests: We apply the MultiSSE and synthesize the running example system (Fig. 1), the *I4Copter* quadcopter control application (Ulbrich et al. 2011), and handcrafted examples to show the working of the prediction. Thereby, we predict the optimization potential based on the analysis result and the models and compare it to the measured end-to-end cycle count.

8.1 Analysis validation

Before synthesizing, we performed tests for the analysis alone, which we present first.

8.1.1 Trampoline conformance tests

Trampoline is an open-source OSEK/VDX and AUTOSAR compliant RTOS implementation (Béchenne et al. 2006). To test the conformance of the MultiSSE, we analyze the provided multi-core related test cases and their applications (without additional timing information). These applications use a broad set of AUTOSAR functionalities including cross-core task activations, locks, alarms, interrupts, timers, and events.

We are able to produce an MSTG for each of those 12 applications. The analysis results reveal neither superfluous locks nor situations resulting in an unexpected deadlock, which is to be expected from the conformance tests. As we do not alter the system, we omit the further synthesis.

³ <https://github.com/luhsra/ara>.

8.1.2 Synthetic Benchmarks

To validate the wider applicability and usefulness of our approach and implementation, we systematically generated synthetic applications. As a benchmark generator, we use the (publicly available) generator presented in Dietrich and Lohmann (2018), which we extended for multi-core systems.

There are six parameters characterizing the generated systems: #cores (2,4,6), #threads (up to 15 per core), #spinlocks (1, 3), #lock-users (2, 6), #cross-core-task-activations (10%), #events (0, 5). We use a pseudo-random number generator with a varying seed to get different but reproducible results. The system generation consists of the following steps: First, we create a directed acyclic thread-dependency graph with #threads nodes. Those threads are mapped onto the specified number of cores. We assign each thread a unique random priority for priority-based scheduling. Then, thread activation is distributed between core-local and cross-core activation following the given percentage. Those threads that are neither activated by a parent thread nor via a cross-core activation are set to be automatically started at system startup. For each spinlock, #lock-users using threads are chosen such that all threads using a lock originate from different cores. A single thread may use multiple locks and there may be multiple threads on a single core accessing different locks. Furthermore, we add additional #events dependencies resulting in tasks waiting for events to be set from other tasks.

We generated 872 applications with varying parameters. Executing the MultiSSE results in graphs with 464 nodes (min: 38, max: 1,613) and 2,837 edges on average (min: 77, max: 8,608). In total 1,890 IPIs were detected as avoidable. From the total of 7,143 attempts to acquire a spinlock, 4,695 are detected to never result in a spinning lock. The run time of the MultiSSE for these examples ranges from 23_s to 7,825 s with a mean of 1,893 s.

Applying randomly generated timing information regarding best-case and worst-case execution times for each ABB results in an average reduction of the MSTG by 30 nodes (6%) and 476 edges (17%). The MultiSSE with timing information has a run time of 23 s to 7,825 s with a mean of 1,908 s.

8.2 Microbenchmarks

According to Fig. 10, we determine the cost of the `ActivateTask` cross syscall including the required IPI as well as the `Get-/ReleaseSpinlock` system calls. We executed all measurements on an ARM Performance Monitor Unit (PMU) of the Raspberry Pi 4B, and we repeated each microbenchmark at least 30 times with a full reboot in between. As expected from such a minimal static system, the cycle count measurements do have a very low standard deviation (average: 1.4%) as there is no interaction with external devices, hence the sample size is large enough. The only exception is `<wait>` with 9% standard deviation, which can be traced back to the cross-core interaction.

In Fig. 12, we show the total cost of an IPI on the sending and receiving side with a constant clock frequency of 250MHz. On the sending side (Fig. 14a), the cost of the

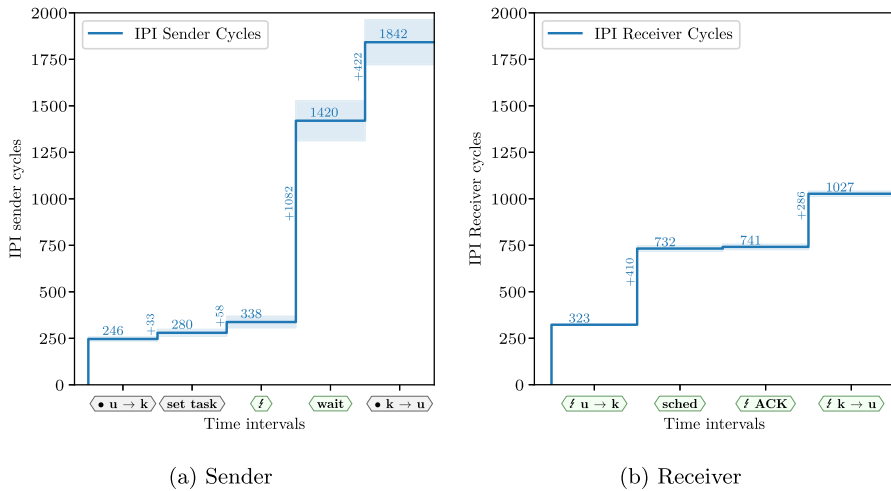


Fig. 12 Amount of cycles needed for each phase of sending (a) and receiving (b) an IPI (see Fig. 10)

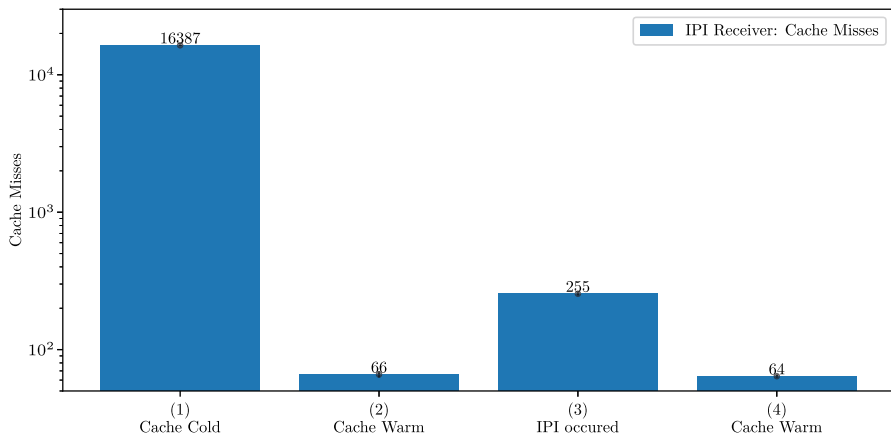


Fig. 13 Cache misses of a data-intensive task after an IPI is received. In the tasks start phase, the cache is cold (1), then warmed up (2). After an IPI, the cache is evicted (3) and has to warm up again (4)

switch from user to kernel mode $\bullet u \rightarrow k$, the scheduler update set task and the IPI trigger f have a constant cycle count, but the largest share is waiting for the IPI to be acknowledged by the receiver wait . The total time from $\bullet u \rightarrow k$ to $\bullet k \rightarrow u$ of the system call in this setup is 1,842 clock cycles, the waiting time in wait alone makes up 1,082 (59%) cycles. The switching between user and kernel mode adds 668 (36%) cycles. In an application where the IPI is unnecessary, the ActivateTask system call could be faster by the time of intervals f + wait (Eq. 1), leading to a system call speedup of 1,140 cycles (62%). On the receiving side (Fig. 14b), the IPI including rescheduling and mode switching, totals to 1,027 cycles possibly interrupting a high-priority task. Additionally, the interrupt causes indirect latencies because the CPU cache is affected by the interrupt routines. The Fig. 13 shows the number of data cache

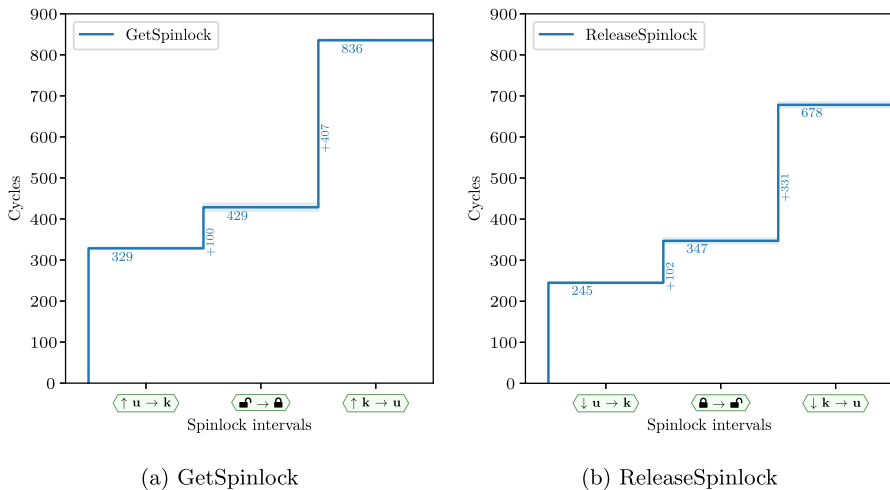


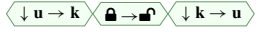
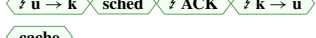
Fig. 14 Amount of cycles needed for each phase of a spinlock (see Fig. 11)

misses during a cache-intensive task while an IPI is received. To minimize the effects of the caching strategy and the 8-way associative cache, the task accesses a specified memory region eight times, the IPI is triggered, and the memory is accessed another eight times to get the cache warm again. In this scenario, where the cache is fully utilized ($16\text{Ki} \times 64\text{Byte} = 1\text{MiB}$), the IPI leads to the eviction of 189 cache lines, which then need to be refilled once the application is continued. The remaining misses ($64 \times 64\text{Byte} = 4\text{KiB}$) are equal to the size of the page table that resides in the cache as well.

Figure 14 shows that the largest part of taking or releasing a spinlock is not the locking operation in the phases $\uparrow$, $\downarrow$ itself, but the system call overhead given by the mode switches in the phases $\uparrow u \rightarrow k$, $\uparrow k \rightarrow u$ and $\downarrow u \rightarrow k$, $\downarrow k \rightarrow u$. Out of the total 836 clock cycles for each GetSpinlock system call, only 100 (12%) are required for the locking operating including error checking and updating the related data structures. Similarly, ReleaseSpinlock takes 678 clock cycles, while the actual system call implementation takes 102 (15%) cycles only. With lock elision applied, we can safely remove both system calls together, thereby freeing 1,514 cycles of unnecessary operations for each pair of GetSpinlock and ReleaseSpinlock. Notably, the ReleaseSpinlock system call is always faster because it requires a GetSpinlock before, leaving a warm cache.

As a result of our measurements, Table 1 contains an overview of the cost of cross-core interactions in our system when relevant data and instructions are not cached, which equals the WCET optimization potential Δt_{opt} per IPI/lock (Eqs. 1, 2, 3). In the following sections, we use these results for further evaluation.

Table 1 WCET optimization potential of cross-core interactions in our implemented RTOS on the Raspberry Pi 4B (Eq. 1, 2, 3)

Phases	Cross-Core Interaction	Δt_{opt} (per IPI/Lock)
	GetSpinlock	1,514 cycles
	ReleaseSpinlock	
	IPI Send	1,140 cycles
	IPI Receive	1,027 cycles
		189 cache misses

8.3 Continuous Interruptions

To test the maximal possible optimization on the receiving side of unnecessary IPIs, we create a test application in which one core is continuously interrupted by IPIs, but none affects the scheduling. While this should not happen in a well-behaved program, such scenarios could be caused in bus networks by faulty nodes (babbling idiots). One ("computing") core runs a high-priority task that is doing a long-running computation. In our test case, the task calculates prime numbers up to 1000. Another ("interrupting") core is constantly trying to activate a low-priority task on the computing core, leading to 672 (unnecessary) IPIs. Due to Eq. 2, this leads to an expected reduction of 690,144 cycles with 1,027 cycles for a single IPI. In this scenario, we measure a reduction of the execution time of the high-priority task from 1,042,771 without optimization to 560,700 cycles with optimization (-46%, 717 cycles per IPI), by automatically removing all 672 unnecessary IPIs. As expected, here, the cost per IPI is lower in the long run than measured in Table 1, due to caching effects.

8.4 The Example Application

Our running example application from Fig. 1 uses the spinlock $S1$ for mutual exclusion during a critical section. The system specification permits three tasks to use the spinlock. Therefore, all those tasks may possibly spin on this lock. Applying MultiSSE without timing information results in an MSTG consisting of 137 vertices connected via 816 edges. Using this graph, we are able to detect that Task T_{11} never gets in conflict with the other tasks acquiring the lock. Hence, those calls may be elided from T_{11} . As it is unclear how long the computation phases of T_{02} and the delta between the activations of T_{02} and T_{01} are, no prediction about the need for an IPI for T_{01} 's activation is possible.

Taking timing information into account (Fig. 2), further reduction of the MSTG is possible. The resulting graph consists of 70 fewer states and 603 fewer transitions. The analysis shows that the activation of T_{01} always occurs during the computation phase of the higher priority T_{02} . Hence, the task activation could be reduced to setting the ready bit. Sending an IPI to trigger the scheduler could be omitted. This results in less priority-inverting activity on core 0 and less interrupt-introduced jitter for the execution time of the high-priority task T_{02} . Furthermore, no other task acquiring

S1 results in no spinning state, and hence, the Spinlock calls are useless and can be removed.

To synthesize this example application correctly, the timing during the execution must be scaled corresponding to the BCET and WCET of each ABB. Therefore, we set the abstract execution time intervals of the example application, e.g., 20–30 units for ABB17 as shown in Fig. 2, to 1000 cycles per unit in the synthesized application (so 1 time unit = 1000 cycles). As described in Sect. 3.1, our model assigns no time to system calls but instead accounts for their time in the surrounding computation ABBs. Hence, in the measurement of the synthesized example application, the computation ABB before each system call contains the (specialized) system call execution time. To measure the execution times as exactly as possible, we add a wait function based on the hardware's clock cycles for each computation ABB, which waits for its BCET. So, for a system call with no execution time (e.g., because the synthesis removes it due to optimization), the execution time of the preceding ABB would equal its BCET. Any optimization in our implementation thus reduces the ABB's WCET.

In Fig. 15, we compare the execution time of the example application (specifically, T11) with and without optimizations. The figure shows, that the `GetSpinlock` and `ReleaseSpinlock` system calls in ABB 10 and ABB 12 can be elided, leading to a reduction of 1,384 cycles. Nevertheless, the execution of these two system calls needs 125 cycles instead of zero. This is due to our wait function that does not need exactly the BCET but a little longer. Our prediction model (Eq. 3) gives 1,514 cycles, which matches (with the 125 cycles added) nearly perfectly. Furthermore, the omitted IPI in ABB 16 leads to 1,004 fewer cycles in comparison to 1,140 predicted cycles given by Eq. 1. Since this IPI is the second one of this task, the measurement is affected by caching and a difference to the model expected. All optimizations combined, the measured execution time of T11 is reduced by 2,381 cycles.

We optimize the other tasks as well, T02 is not interrupted by an IPI anymore and is 900 cycles faster (calculated: 1,027 cycles, Eq. 2). In both tasks, T01 and T21, the Spinlock system calls are elided, reducing the execution time by 1,415 cycles (calculated: 1,514 cycles, Eq. 3).

In summary, all measured execution times of the tasks of the example application are reduced in a range fitting to the formulas.

8.5 The *I4Copter*

The *I4Copter* is an embedded real-time flight controller software formerly designed for a single-core platform, that was kindly provided to us by Ulbrich et al. (2011). For this evaluation, we adapted the task set to distribute the two major components *sensor gathering* and *actuation* onto two cores. Those two parts work mostly without interference. The only shared resource is the SPI bus. Therefore, we introduced a spinlock to mutually exclude access from the two cores. Figure 16 depicts the components and their interworking.

Execution of the MultiSSE on an Intel i5-6400 quad-core system with 32 GiB of main memory takes 12.79 s. Applying MultiSSE generates an MSTG with 294 vertices and 2,143 edges. Applying timing information reduces it by 140 vertices (48%) and

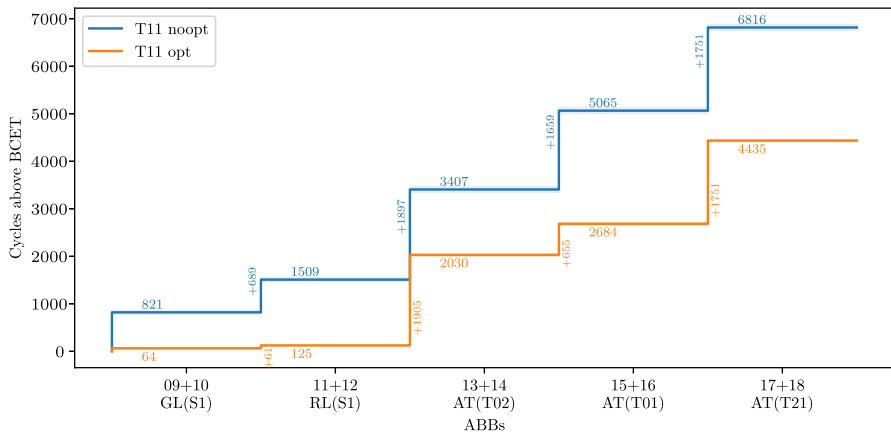


Fig. 15 Optimized execution time of the example application Task T11. The execution time is reduced by Lock Elision in ABB10/12 and omitting the IPI in ABB16. The BCET is subtracted on the y-axis to make the difference between the times more visible

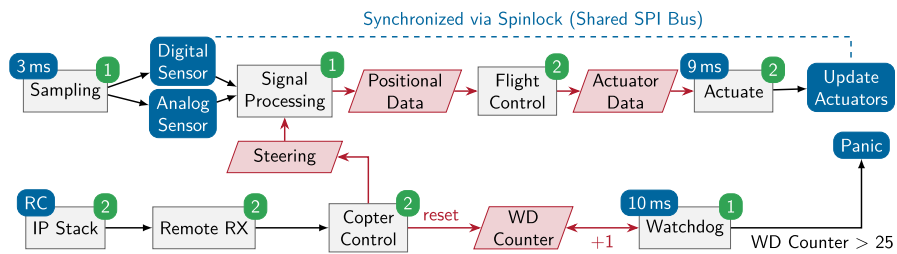


Fig. 16 14Copter flight-controller application (adapted from Ulbrich et al. (2011)). Eight tasks (grey) are mapped on two cores (green) (Color figure online)

1,496 edges (70%). Additionally, the graph shows that the spinlock, introduced to guard the SPI bus shared between the two cores, will never result in a spinning state.

We have synthesized the spinlock-free optimized system. The analysis shows that four locking calls in the sampling task are avoidable, leading to a prediction of $3,028 - 2 \cdot 125 = 2,778$ that can be removed. Our measurement gives 2,748 (14% of the task's execution time) cycles, fitting to the expected value, while removing the Get/ReleaseSpinlock pair in the longer running actuating task on the other core (expected cycle reduction: 1,389 cycles) leads to 1,325 fewer cycles (4% of the task's execution time), also fitting to the reduction.

9 Discussion

The biggest caveat of the MultiSSE is the exponential growth when calculating every possible system state. We do mitigate this by aggressively merging or omitting states whenever possible:

- We use ABBs as underlying control-flow representation. They effectively reduce the control-flow states in contrast to basic blocks or plain instruction by subsuming everything in one block that is not relevant for the analysis.
- We merge single-core states, whenever possible. If the analysis determines that an SP does not modify the local-core state, it merges it with the prior state (and skips further evaluations). For example, a `GetSpinlock` system call modifies a global context and, thus, never a LABSS.
- We build an SP only when it is of interest. An essential part of the analysis consists of the calculation of the minimal set of SPs required to still capture the effects of all cross syscalls.

Since the analysis belongs to the building process and thus is executed only a few times and given the actual results from our evaluation, we consider the analysis overhead as reasonable. Further reduction will be possible by incorporating more knowledge about the real-time control systems (RTCSs) into our analysis. The MultiSSE already utilizes worst/best-case execution times to determine impossible SPs. In future work, additional sources should be exploited to further reduce the SPs amount: One example is to further explicit ordering of semantically dependent system calls on different cores by incorporating their temporal logic of actions (Lamport 1994). Another possibility is that application developers provide annotations or explicit barriers as SP to bring all cores in sync at suitable points within the application's logic.

For timing calculations, the MultiSSE requires a stricter set of real-time properties like bounded loops or falls back to disabling them. Note, however, that having timing information is optional – it just improves analysis results and time. Furthermore, the availability of precise timing information is not an all-or-nothing requirement. If the timing analysis fails for some states, it can still be employed for the states after the next SP.

While we have not formally proven MultiSSE's soundness, we are convinced that it holds true. The analysis result is an over-approximation in all cases, making too optimistic optimizations impossible. Assuming the SSE's soundness, we only have to demonstrate soundness for cross-core interactions added by MultiSSE. The OS specification and our system model clearly restrict cross-core interactions to a finite set of system calls and interrupts. Under the assumption of known system-call arguments (already needed for the SSE), the MultiSSE is sound by calculating the superset of all possible effects of all system calls and interrupts. The MultiSSE aims not to be correct, in doubt, it always chooses the pessimistic approach, which – in the worst case – may result in missing optimizations.

Due to the fact, that the optimization result, the actual WCET reduction, depends on the hardware and the RTOS and application implementation, we developed cross-core interaction cost models independent of those factors and showed its working. In our synthesis, we base on a system that is already statically tailored towards the application, but we can see significant potential for improvement with our proposed specializations. We expect other RTOS implementations on other platforms to be affected similarly. For example, including mode switches on an x86 system, the IPI sender cost is over 2000 cycles and the receiver cost is even higher at over 2600 cycles (Kaffes et al. 2019).

Regarding Spinlocks, the number of switches between kernel and user mode for Spinlocks could be reduced if the system would implement a *futex* mechanism (Hubertus Franke 2002), so the kernel can dynamically decide to remain in user mode during the locking operation. In the WCET calculation, however, this would require the assumption of taking the slow path (i.e., a mode switch again). Therefore, statically knowing about unnecessary locks can lead to a WCET reduction, while other spinlock implementations (that implement another run-time behavior) cannot.

RTOS implementations without a separate kernel mode benefit less from our specializations but they still do. Rommel et al. showed that taking a lock within the Linux kernel on a single-core system results in a measurable performance impact (Rommel et al. 2019). That coincides with our results that show a run time of 100 cycles for executing the locking operation alone (phases  → ,  → ). Additionally, unnecessary and, hence, guaranteed contention-free lock taking potentially may – depending on the RTOS-internal implementation and the target architecture – trigger costly cache-coherence messages between cores (Glew and Hwu 1990).

10 Related Work

System-call specialization based on static analysis is not new. For dynamic embedded systems, Bertran et al. introduced a global view of the interactions between the application and operating system by constructing a global control-flow graph (CFG) (Bertran et al. 2006) that connects the system-call entry point to their corresponding application call sites. With that, they can eliminate dead code of uncalled system call functions, which reduces code size. Rajagopalan et al. (2005) try to detect and remove unused kernel parts by statically analyzing and rewriting the kernel binary. In contrast to this work, both analyses respect neither OS semantics nor multiple cores.

The first respecting OSEK semantics are Barthelmann (2002). They use it to reduce the costs of task context switching. A static analyzer constructs an inference graph describing the mutual preemptions of basic blocks, which allows to facilitate an optimized inter-task register allocation and static context switch code generation. In contrast to this work, the analysis is flow-insensitive. Hence, the influence graph includes superfluous combinations which could be avoided by considering task activations. Also, since the work uses OSEK semantics, it targets only single-core systems.

Also, specializations exist for general-purpose operating systems. Pu et al. developed the Synthesis kernel, which includes a code synthesizer that produces application-dependent optimized code paths at run time for frequently called system calls (Pu et al. 1988). They provide manually optimized templates which are filled out during run time which leads to an overall speed-up. In contrast to the dynamic Synthesis system, our approach takes place ahead of time, which also allows system call removals.

With SWAN (Schuster et al. 2019), Schuster et al. developed a whole-system WCET analyzer that builds a state transition graph to achieve tighter bounds for the WCET. However, SWAN is not able to model multi-core systems and does not aim for specialization.

Dietrich et al. introduced the SSE (Dietrich et al. 2015) which we used as a base for the MultiSSE. While we share the algorithms for the single-core analysis, we adapted

the data structures and functions to work for multiple cores. They synthesized SSE-optimized systems using *dOSEK* (Hoffmann et al. 2015), which we extended for multi-core support as well. We additionally introduced the SPs mechanism and completely changed the interrupt handling. As a follow-up analysis for the SSE, Dietrich et al. further introduced another method – the System-State Flow – to construct (much faster) a more coarse-grained state graph that can be used for similar specializations (Dietrich et al. 2017). While this greatly reduces the algorithm’s execution time, it does this at the cost of state precision. For the MultiSSE, we need the full set of possible states to construct the full set of SPs, which is why we chose to build upon the SSE.

Regarding the use of spinlocks in embedded real-time systems, Wieder et al. demonstrate the problematic semantics of spinlocks for schedulability analyses of multi-core real-time applications (Wieder and Brandenburg 2013). With our approach, we are able to elide the locks from the application or derive the order of lock acquisitions to further improve those analyses.

With special regard to multi-core analysis, Mittermayr et al. modeled multi-core synchronization patterns with the help of Kronecker-algebra, a method to model parallel executing automata in form of a matrix (Mittermayr and Blieberger 2016, 2021). It represents all possible intersections, which, with knowledge of barriers (semaphores), is reducible to actual possible paths and lazily computable. While the resulting graph models cross-core interactions in a flow-sensitive manner, it does concentrate on barriers only and is not modeling other RTOS primitives. Furthermore, the approach is not used for system-call specialization, but for deadlock detection and WCET analysis. Haur et al. use high-level colored time Petri nets to model a multi-core application for AUTOSAR including the RTOS code (Haur et al. 2021, 2022). With their approach, they were able to find incorrect locking patterns within the RTOS code by formal analysis. However, in contrast to our approach, they do not focus on unnecessary locks and need to map manually from the application code to the model. Miné et al. extended *Astreé*, an abstract interpreter to support multi-core programs to which Kästner et al. later added support for AUTOSAR applications (Miné 2012; Kaestner et al. 2017). While they are doing a flow-sensitive analysis and taking locks into account, they collect all cross-core interactions in a flow-insensitive fact pool and iterate the analysis until it becomes stable. While their approach scales better and does not concentrate on operating system facts only, they are neither flow-sensitive between cores nor take their possible orders into account. Schwarz et al. transform multi-core programs into a set of constraints with side effects to reason over all local and global flow paths (Schwarz et al. 2021). In contrast to the MultiSSE their approach detects run-time errors regarding global data usage and works for POSIX threads.

Regarding deadlock detection, Engler et al. searched for them via an interprocedural flow-sensitive but unsound static analysis (Engler and Ashcraft 2003), which – in contrast to our approach – provides false positives and aims application development. Kroening et al. present a method for sound deadlock detection for the C language using pthreads (Kroening et al. 2016). For that, they build a similar control flow graph but construct a special lock graph, which does not model arbitrary thread interactions.

In the topic of cross-core interaction cost measurements, Kaffes et al. measured IPI cycles for scheduling in networks (Kaffes et al. 2019). They provide measurements

on an x86 system and implement centralized scheduling policies to reduce network latencies but do not model the cost of IPIs or spinlocks systematically.

11 Conclusions

With MultiSSE, we presented an algorithm that creates a multi-core state transition graph of all possible cross-core interactions in a partitioned real-time system. It works by traversing the application's CFG and iteratively building a graph of all core-local RTOS states which it connects with explicit synchronization points for a cross-core interaction.

To reduce the number of synchronization points, the algorithm can additionally make use of externally provided timing information. With that, we could reduce the MSTG vertices up to 48% and edges up to 70%.

With the help of the MSTG, we were able to extract the necessary information for cross-core system-call specializations: We showed the possibility of removing superfluous (not spinning) locks and locking system calls and the possibility of avoiding IPIs that trigger an unnecessary reschedule. In addition to this, the MSTG provides all necessary information for single-core system-call specializations as described in prior works. To be able to get a prediction of the cost savings, we developed two cost models showing the different system call phases and measured real numbers for these phases on an ARM platform.

We have validated our approach with unit tests as well as 872 synthetically generated applications, showing 66% of attempts to take a spinlock as useless and a total of 1,890 of IPIs as avoidable. Additionally, we applied our approach to an adapted version of the *I4Copter*, an embedded real-time flight controller, and were able to reduce its run time by 14% by classifying the existing lock as useless and automatically omitting it. Finally, we showed the maximal optimization potential, where we could improve the run time of the highest priority task by 46%.

Acknowledgements We want to thank our reviewers for their thorough reviews. This work was partly funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – 391395160 and 468988364.

Funding Open Access funding enabled and organized by Projekt DEAL.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

AEEC (2015) Avionics Application Software Standard Interface (ARINC Specification 653P1-4)

- AUTOSAR (2013) Specification of operating system (version 5.1.0). Technical report, Automotive Open System Architecture GbR
- Barthelmann V (2002) Inter-task register-allocation for static operating systems. In: Proceedings of the Joint Conference on Languages, Compilers and Tools for Embedded Systems (LCTES/SCOPES '02), pp. 149–154. ACM Press, New York, NY, USA. <https://doi.org/10.1145/513829.513855>
- Béchenne J-L, Briday M, Faucou S, Trinquet Y (2006) Trampoline: An OpenSource implementation of the OSEK/VDX RTOS specification. In: IEEE Conference on Emerging Technologies and Factory Automation, 2006. ETFA '06., pp. 62–69. IEEE Computer Society Press, Washington, DC, USA. <https://doi.org/10.1109/ETFA.2006.355432>
- Bertran R, Gil M, Cabezas J, Jimenez V, Vilanova L, Morancho E, Navarro N (2006) Building a global system view for optimization purposes. In: Proceedings of the 2nd Workshop on the Interaction Between Operating Systems and Computer Architecture (WIOSCA '06). IEEE Computer Society Press, Washington, DC, USA
- Cooling J (2003) Software engineering for real-time systems. Addison-Wesley, New York
- Dietrich C (2019) Interaction-aware analysis and optimization of real-time application and operating system. PhD thesis, Leibniz Universität Hannover. <https://doi.org/10.15488/7253>. <https://www.repo.uni-hannover.de/handle/123456789/7306>
- Dietrich C, Hoffmann M, Lohmann D (2017) Global optimization of fixed-priority real-time systems by RTOS-aware control-flow analysis. *ACM Trans Embedded Comput Syst* 16(2):35–13525. <https://doi.org/10.1145/2950053>
- Dietrich C, Hoffmann M, Lohmann D (2015) Cross-kernel control-flow-graph analysis for event-driven real-time systems. In: Proceedings of the 2015 ACM SIGPLAN/SIGBED Conference on Languages, Compilers and Tools for Embedded Systems (LCTES '15). ACM Press, New York, NY, USA. <https://doi.org/10.1145/2670529.2754963>
- Dietrich C, Lohmann D (2017) OSEK-V: Application-specific RTOS instantiation in hardware. In: Proceedings of the 2017 ACM SIGPLAN/SIGBED Conference on Languages, Compilers and Tools for Embedded Systems (LCTES '17). ACM Press, New York, NY, USA. <https://doi.org/10.1145/3078633.3078637>
- Dietrich C, Lohmann D (2018) Semi-extended tasks: Efficient stack sharing among blocking threads. In: Altmeyer, S. (ed.) Proceedings of the 39th IEEE Real-Time Systems Symposium 2018. IEEE Computer Society Press, Nashville, Tennessee, USA. <https://doi.org/10.1109/RTSS.2018.00049>
- Engler D, Ashcraft K (2003) Racex: effective, static detection of race conditions and deadlocks. In: Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP '03), pp. 237–252. ACM Press, New York, NY, USA. <https://doi.org/10.1145/945445.945468>
- Entrup G, Fiedler B, Lohmann D (2023) MultiSSE: Static syscall elision and specialization for event-triggered multi-core RTOS. In: Proceedings of the 29th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS'23)
- Ermedahl A, Gustafsson J, Lisper B (2011) Deriving wcet bounds by abstract execution. In: ECRTS 2011
- Fiedler B, Entrup G, Dietrich C, Lohmann D (2021) ARA: Static initialization of dynamically-created system objects. In: Proceedings of the 27th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS'21), pp. 400–412. <https://doi.org/10.1109/RTAS52030.2021.00039>
- Glew A, Hwu W-M (1990) Snoopy cache test-and-test-and-set without excessive bus contention. *SIGARCH Comput Archit News* 18(2):25–32. <https://doi.org/10.1145/88237.88240>
- Hammarlund P, Crossland JB, Kaushik SD, Aggarwal A (2003) Inter-processor Interrupts. US Patent 8,984,199 B2. <https://patents.google.com/patent/US8984199B2/en>
- Haur I, Béchenne J-L, Roux OH (2022) Formal verification of the inter-core synchronization of a multi-core rtos kernel. In: Riesco A, Zhang M (eds) Formal methods and software engineering. Springer, Berlin, pp 140–155
- Haur I, Béchenne J-L, Roux OH (2021) Formal schedulability analysis based on multi-core RTOS model. In: 29th International Conference on Real-Time Networks and Systems. RTNS'2021, pp. 216–225. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/3453417.3453437>
- Hofer W, Lohmann D, Scheler F, Schröder-Preikschat W (2009) Sloth: Threads as interrupts. In: Proceedings of the 30th IEEE International Symposium on Real-Time Systems (RTSS '09), pp. 204–213. IEEE Computer Society Press, Washington. <https://doi.org/10.1109/RTSS.2009.18>
- Hoffmann M, Lukas F, Dietrich C, Lohmann D (2015) dOSEK: The design and implementation of a dependability-oriented static embedded kernel. In: Proceedings of the 21st IEEE International Sym-

- posium on Real-Time and Embedded Technology and Applications (RTAS '15), pp. 259–270. IEEE Computer Society Press, Washington, DC, USA. <https://doi.org/10.1109/RTAS.2015.7108449>
- Holsti N (2008) Computing time as a program variable: a way around infeasible paths. In: Kirner, R. (ed.) 8th International Workshop on Worst-Case Execution Time Analysis (WCET'08). OpenAccess Series in Informatics (OASISs), vol. 8. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany. <https://doi.org/10.4230/OASISs.WCET.2008.1660> . also published in print by Austrian Computer Society (OCG) with ISBN 978-3-85403-237-3. <http://drops.dagstuhl.de/opus/volltexte/2008/1660>
- Hubertus Franke MK (2002) Rusty Russell: Fuss, futexes and furwicks: Fast userlevel locking in Linux. In: Hutton, A.J., Donovan, S., Ross, C.C. (eds.) Proceedings of the Ottawa Linux Symposium, pp. 479–495
- Kaestner D, Miné A, Schmidt A, Hille H, Mauborgne L, Wilhelm S, Rival X, Feret J, Cousot P, Ferdinand C (2017) Finding all potential run-time errors and data races in automotive software. SAE International, Detroit. <https://doi.org/10.4271/2017-01-0054>
- Kaffes K, Chong T, Humphries JT, Belay A, Mazières D, Kozyrakis C (2019) Shinjuku: Preemptive scheduling for μ second-scale tail latency. In: 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19), pp. 345–360. USENIX Association, Boston, MA. <https://www.usenix.org/conference/nsdi19/presentation/kaffes>
- Khachiyan LG (1979) A Polynomial Algorithm in Linear Programming 244:1093–1096
- Kroening D, Poetzl D, Schrammel P, Wachter B (2016) Sound static deadlock analysis for c/pthreads. In: Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering. ASE 2016, pp. 379–390. Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/2970276.2970309>
- Lakshmanan K, Bhatia G, Rajkumar R (2011) AUTOSAR extensions for predictable task synchronization in multi-core ECUs. In: Proceedings of the SAE 2011 World Congress
- Lakshmanan K, Kato S, Rajkumar RR (2010) Scheduling parallel real-time tasks on multi-core processors. In: Proceedings of the 31st IEEE International Symposium on Real-Time Systems (RTSS '10), pp. 259–268. IEEE Computer Society Press, Washington, DC, USA
- Lamport L (1994) The temporal logic of actions. ACM Trans Program Lang Syst 16(3):872–923. <https://doi.org/10.1145/177492.177726>
- Lattner C, Adve V (2004) LLVM: A compilation framework for lifelong program analysis & transformation. In: Proceedings of the 2004 International Symposium on Code Generation and Optimization (CGO'04), pp. 75–86. IEEE Computer Society Press, Washington, DC, USA. <https://doi.org/10.1109/CGO.2004.1281665>
- Leyva-del-Foyo LE, Mejia-Alvarez P, Niz D (2006) Predictable interrupt management for real time kernels over conventional PC hardware. In: Proceedings of the 12th IEEE International Symposium on Real-Time and Embedded Technology and Applications (RTAS '06), pp. 14–23. IEEE Computer Society Press, Los Alamitos, CA, USA. <https://doi.org/10.1109/RTAS.2006.34>
- Marwedel P (2006) Embedded system design. Springer, Heidelberg
- Miné A (2015) AstréeA: A static analyzer for large embedded multi-task software. In: Proc. of the 16th International Conference on Verification, Model Checking, and Abstract Interpretation (VMCAI'15). Lecture Notes in Computer Science (LNCS), vol. 8931, p. 3. Springer, Berlin
- Miné A (2012) Static analysis of run-time errors in embedded real-time parallel c programs. Log Methods Comput Sci. [https://doi.org/10.2168/lmcs-8\(1:26\)2012](https://doi.org/10.2168/lmcs-8(1:26)2012)
- Mittermayr R, Blieberger J (2016) A generic graph model for wcet analysis of multi-core concurrent applications. J Softwa Eng Appl 9:182–198. <https://doi.org/10.4236/jsea.2016.95015>
- Mittermayr R, Blieberger J (2021) Deadlock and wcet analysis of barrier-synchronized concurrent programs. Computing 103:749–770. <https://doi.org/10.1007/s00607-017-0555-8>
- OSEK/VDX Group (2005) Operating system specification 2.2.3. Technical report, OSEK/VDX Group. <http://portal.osek-idx.org/files/pdf/specs/os223.pdf>, visited 2014-09-29
- Pu C, Massalin H, Ioannidis J (1988) The Synthesis kernel. Comput Syst 1(1):11–32
- Rajagopalan M, Debray SK, Hiltunen MA, Schlichting RD (2005) Automatic operating system specialization via binary rewriting
- Rinard M (2001) Analysis of multithreaded programs. In: Cousot P (ed) Static analysis. Springer, Berlin, pp 1–19
- Rommel F, Dietrich C, Rodin M, Lohmann D (2019) Multiverse: Compiler-assisted management of dynamic variability in low-level system software. In: Fourteenth EuroSys Conference 2019 (EuroSys '19). ACM Press, New York, NY, USA. <https://doi.org/10.1145/3302424.3303959>

- Scheler F, Schröder-Preikschat W (2010) The RTSC: Leveraging the migration from event-triggered to time-triggered systems. In: Proceedings of the 13th IEEE International Symposium on Object-Oriented Real-Time Distributed Computing (ISORC '10), pp. 34–41. IEEE Computer Society Press, Washington, DC, USA. <https://doi.org/10.1109/ISORC.2010.11>
- Schuster S, Wägemann P, Ulbrich P, Schröder-Preikschat W (2019) Proving real-time capability of generic operating systems by system-aware timing analysis. In: 2019 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS), pp. 318–330. <https://doi.org/10.1109/RTAS.2019.00034>
- Schwarz M, Saan S, Seidl H, Apinis K, Erhard J, Vojdani V (2021) Improving thread-modular abstract interpretation. In: Drăgoi C, Mukherjee S, Namjoshi K (eds) Static analysis. Springer, Chicago, pp 359–383
- Ulbrich P, Kapitza R, Harkort C, Schmid R, Schröder-Preikschat W (2011) I4Copter: An adaptable and modular quadrotor platform. In: Proceedings of the 26th ACM Symposium on Applied Computing (SAC '11), pp. 380–396. ACM Press, New York, NY, USA
- Wägemann P, Dietrich C, Distler T, Ulbrich P, Schröder-Preikschat W (2018) hole-system worst-case energy-consumption analysis for energy-constrained real-time systems. In: Altmeyer, S. (ed.) Proceedings of the 30th Euromicro Conference on Real-Time Systems 2018. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik. <https://doi.org/10.4230/LIPIcs.ECRTS.2018.24>
- Wieder A, Brandenburg BB (2013) On spin locks in AUTOSAR: Blocking analysis of FIFO, unordered, and priority-ordered spin locks. In: Proceedings of the 34th IEEE International Symposium on Real-Time Systems (RTSS '13), pp. 45–56. IEEE Computer Society Press, USA. <https://doi.org/10.1109/RTSS.2013.13>
- Zephyr Project homepage. <https://www.zephyrproject.org/>

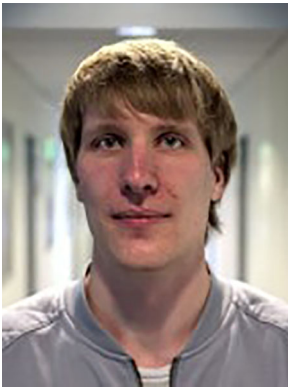
Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Gerion Entrup received his bachelor's (2013, 2014) and master's degree (2017) from Leibniz Universität Hannover, Germany. After that, he joined the System Research and Architecture (SRA) group at Leibniz Universität Hannover, where he worked as Ph.D. student in the research field of operating-system aware analysis especially aiming embedded real-time systems.



Andreas Kässens received a Bachelor's degree from Ostfalia University of Applied Sciences in Wolfenbüttel, Germany in 2020, followed by a Master's degree from Leibniz Universität Hannover in Germany in 2023. He is currently a Ph.D. candidate in the Systems Research and Architecture group at Leibniz Universität Hannover, focusing his research on Real-Time Operating Systems for embedded devices.



Björn Fiedler graduated from Leibniz Universität Hannover, Germany, in 2015 with a bachelor's and in 2017 with a master's degree in computer science. In 2018, he joined the System Research and Architecture (SRA) group at Leibniz Universität Hannover, where he focused his research on the specialization of embedded systems and received his Ph.D. (Dr.-Ing.) in 2023. Since 2023, he is a post-doc at SRA. He continues his work on automated specialization of embedded real-time systems.



Daniel Lohmann received his diploma (Dipl.-Inform.) from Universität Koblenz, Germany in 2002. In 2003 he joined the Systems Software Group at Friedrich-Alexander-Universität Erlangen-Nürnberg, where he received his PhD (Dr.-Ing.) in 2009 and his *venia legendi* (Dr.-Ing. habil.) in 2014. Since 2017 he is full professor and head of the Systems Research and Architecture (SRA) group at Leibniz Universität Hannover. Prof. Lohmann's research and teaching activities center around the architecture of computing systems: From hardware over system software up to languages and compilers. His work targets fundamental research on system software for computing systems that have to cope with strong demands regarding nonfunctional properties, such as noise, timeliness, robustness and scalability. Focus is on constructive methods for the design and development of versatile (real-time) operating systems that provide an extreme degree of (automatic) tailorability towards concrete application requirements and hardware properties.